A

Mini Project Report

On

Interactive Animated Dashboard for Child Sexual Offence (POCSO) Case Data in India using Python and Streamlit

Submitted to DR.K.V.SUBBA REDDY INSTITUTE OF TECHNOLOGY

In Partial Fulfillment of the requirements for the Award of Degree of

BACHELOR OF TECHNOLOGY IN

ARTFICIAL INTELLIGENCE

Submitted By

M.BUNNY BABU	**(24FH1A3194)**
N.RAVINDRA RAJU	**(24FH1A3197)**
T.MAHENDRA	**(24FH1A31C6)**
T.SURESH	**(24FH1A31C7)**
Y.KUMARA SWAMY	**(24FH1A31D1)**

Under the Esteemed guidance of

Dr. B.MAHESH

Associate Professor, Department of CSE

Department of AI

DR.K.V.SUBBA REDDY INSTITUTE OF TECHNOLOGY

UGC AUTONOMOUS

(Approved by AICTE, NEW DELHI, Affiliated to JNTUA, Anantapur)

2026-2027

DR.K.V.SUBBA REDDY INSTITUTE OF TECHNOLOGY

UGC AUTONOMOUS

(Accredited by NBA, Approved by AICTE NEW DELHI, Affiliated to JNTUA, Anantapur)

Department of AI

CERTIFICATE

This is to certify that the project entitled **"Interactive Animated Dashboard for Child Sexual Offence (POCSO) Case Data in India using Python and Streamlit"** is a bonafide work carried out by

M.BUNNY BABU	**(24FH1A3194)**
N.RAVINDRA RAJU	**(24FH1A3197)**
T.MAHENDRA	**(24FH1A31C6)**
T.SURESH	**(24FH1A31C7)**
Y.KUMARA SWAMY	**(24FH1A31D1)**

in partial fulfillment of the requirement for the award of the degree of **BACHELOR OF TECHNOLOGY** in **AI** from Dr.K.V.Subba Reddy Institute of Technology, Kurnool, under our guidance and supervision.

The results presented in this Mini project have been verified and are found to be satisfactory. The results embodied in this Mini project have not been submitted to any other university for the award of any other degree or diploma.

Guide **Head of the Department**

DECLARATION

This is to certify that the work reported in the present Mini project entitled “Interactive Animated Dashboard for Child Sexual Offence (POCSO) Case Data in India using Python and Streamlit” is a record of bonafide work done by us in the Department of AI, Dr.K.V.Subba Reddy Institute of Technology, Kurnool. The report is based on the project work done entirely by us and not copied from any other source. We submit our project for further development by any interested students who share similar interests to improve the project in the future.

The results embodied in this Mini project report have not been submitted to any other University or Institute for the award of any degree or diploma to the best of our knowledge and belief.

M.BUNNY BABU	**(24FH1A3194)**
N.RAVINDRA RAJU	**(24FH1A3197)**
T.MAHENDRA	**(24FH1A31C6)**
T.SURESH	**(24FH1A31C7)**
Y.KUMARA SWAMY	**(24FH1A31D1)**

ACKNOWLEDGMENT

We are extremely grateful to Dr. K Sree Latha, Principal and H. Ateeq Ahmed, HOD, Department of AI, Dr.K.V.Subba Reddy Institute of Technology for their constant support.

We are extremely thankful to Dr. B.MAHESH, Associate Professor, Department of CSE, Internal Guide, for his constant guidance, encouragement and moral support throughout the project.

We will be failing in duty if we do not acknowledge with grateful thanks the authors of the references and other literature referred to in this project, and the open-source communities behind Python, Streamlit, pandas, NumPy, SciPy, Matplotlib, Seaborn and Plotly.

We express our thanks to all staff members and friends for all the help and co-ordination extended in bringing out this project successfully in time.

Finally, we are very much thankful to our parents who guided us at every step.

M.BUNNY BABU **(24FH1A3194)**

N.RAVINDRA RAJU **(24FH1A3197)**

T.MAHENDRA **(24FH1A31C6)**

T.SURESH **(24FH1A31C7)**

Y.KUMARA SWAMY **(24FH1A31D1)**

CONTENTS

TOPIC	PAGE NO.
1. ABSTRACT	**i**
LIST OF FIGURES	**ii**
LIST OF TABLES	**iii**
2. INTRODUCTION	**1**
2.1 BACKGROUND	2
2.2 MOTIVATION	2
2.3 PROBLEM STATEMENT	3
2.4 OBJECTIVES OF THE PROJECT	3
2.5 SCOPE OF THE PROJECT	4
3. LITERATURE SURVEY / EXISTING SYSTEM	**5**
3.1 EXISTING DATA VISUALIZATION TOOLS	6
3.2 LIMITATIONS OF EXISTING SYSTEM	7
3.3 PROPOSED SYSTEM	7
4. TECHNOLOGIES AND TOOLS USED	**9**
4.1 PYTHON	10
4.2 STREAMLIT	10
4.3 PANDAS	10
4.4 NUMPY	10
4.5 MATPLOTLIB / SEABORN / PLOTLY	11
5. SYSTEM REQUIREMENTS	**12**
5.1 HARDWARE REQUIREMENTS	13
5.2 SOFTWARE REQUIREMENTS	13
6. DATASET DESCRIPTION	**14**
6.1 DATASET SOURCE	15
6.2 DATASET ATTRIBUTES	15
6.3 DATA TYPES	15
6.4 DATA PREPROCESSING	15
6.5 DATA CLEANING	16
7. IMPLEMENTATION	**18**
7.1 PROJECT STRUCTURE	19
7.2 DATA LOADING	20
7.3 DATA PREPROCESSING	20
7.4 VISUALIZATION DEVELOPMENT	21
7.5 STREAMLIT INTERFACE DEVELOPMENT	21
7.6 INTERACTIVE FILTERS	21
7.7 DASHBOARD INTEGRATION	22
8. DATA VISUALIZATION	**23**
8.1 BAR CHART	24
8.2 LINE CHART	25

8.3 PIE CHART 25
8.4 SCATTER PLOT 26
8.5 HISTOGRAM 26
8.6 OTHER VISUALIZATIONS USED 27
9. RESULTS AND DISCUSSION 31
9.1 DASHBOARD SCREENSHOTS 32
9.2 VISUALIZATION RESULTS 32
9.3 ANALYSIS OF RESULTS 33
9.4 KEY INSIGHTS 34
10. ADVANTAGES AND LIMITATIONS 35
10.1 ADVANTAGES 36
10.2 LIMITATIONS 36
11. CONCLUSION 38
12. FUTURE ENHANCEMENT 40
13. REFERENCES 42
14. APPENDIX 43
A SOURCE CODE 44
B SAMPLE DATASET 54
C ADDITIONAL SCREENSHOTS 55
15. GITHUB REPOSITORIY
https://github.com/BunnyBabu-oct21/POCSO-Cases-in-India---Dashboard

ABSTRACT

Data about offences against children in India is published every year by the National Crime Records Bureau, but it is usually consumed as static tables and single charts. This project develops an interactive and animated dashboard that makes one verified dataset of cases registered under the Protection of Children from Sexual Offences (POCSO) Act, 2012 easier to explore, using only the Python ecosystem.

The dashboard is built with Streamlit and uses pandas, NumPy and SciPy for data preparation and statistics, and Plotly, Seaborn and Matplotlib for visualization. The bundled dataset has 12 records for 2020–2023: all-India totals for 2020–2022 and state-level values for five states in 2021, Uttar Pradesh in 2022 and Karnataka in 2023. A sidebar offers filters for year, geography, offence category and case-count range. The application provides bar, line, pie, donut, radar, histogram, sunburst, time-series and scatter charts and a heatmap; selected charts are animated with adjustable speed, and key-indicator cards count up on load. There is no file-upload option.

Registered cases at the national level rise from 47,221 in 2020 to 63,414 in 2022 (34.3% in two years, about 15.9% per year), the five listed states account for 49.5% of the 2021 total, and girls make up about 97% of victims. The report stresses that these are registered cases, not a measure of incidence, and labels all derived values and data gaps. The result is a lightweight, open-source and extensible tool for studying child-protection data.

Keywords: Data Visualization, Streamlit, Python, Plotly, POCSO Act, NCRB, Interactive Dashboard, Animation.

LIST OF FIGURES

S. NO.	FIG. NO.	DESCRIPTION	PAGE NO.
1	7.1	System architecture of the dashboard	19
2	7.2	Data processing flow	20
3	8.1	Bar charts: (a) state-level cases in 2021; (b) all-India totals 2020–2022	24
4	8.2	Line chart of registered cases by geography (log scale)	25
5	8.3	Pie chart: composition of 2021 registered cases	26
6	8.4	Scatter plots: (a) rank vs cases for 2021; (b) first vs latest reported year	26
7	8.5	Histogram with scaled kernel density estimate of state-level case counts	27
8	8.6	Donut chart: victim gender split, 2017–2020 (percentages, not counts)	28
9	8.7	Radar chart: registered cases by state, 2021	28
10	8.8	Sunburst chart (shown as nested rings): year → component	29
11	8.9	Time series of all-India registered cases with year-on-year change and linear trend	29
12	8.10	Seaborn heatmap of state-level cases by year	30
13	9.1	Schematic layout of the dashboard (not a screenshot)	32
14	C.1	Sidebar filters and animation controls	55
15	C.2	Dashboard header and key-indicator cards	56
16	C.3	Overview tab: animated bar chart and donut chart	56
17	C.4	Overview tab: composition (pie chart and sunburst)	57
18	C.5	Trends tab: animated line chart	57
19	C.6	Trends tab: linear fit and change table	58
20	C.7	Geography tab: radar chart and seaborn heatmap	58
21	C.8	Distribution & relationships tab: histogram	59
22	C.9	Distribution & relationships tab: scatter plots	59
23	C.10	Data & notes tab: records in view	60
24	C.11	Data & notes tab: about this data	60

LIST OF TABLES

S. NO.	TABLE NO.	DESCRIPTION	PAGE NO.
1	3.1	Common data visualization tools	6
2	3.2	Related literature and its influence on this project	6
3	3.3	Comparison of existing approaches and the proposed system	8
4	4.1	Libraries and their role in the project	11
5	5.1	Recommended hardware configuration	13
6	5.2	Software configuration	13
7	6.1	Attributes of the dataset	15
8	6.2	Data types after loading	15
9	6.3	Coverage of case-count records	16
10	6.4	Descriptive statistics of the case-count records	16
11	7.1	Project files	19
12	7.2	Sidebar controls	22
13	7.3	Dashboard tabs and the charts they contain	22
14	8.1	Summary of chart types	24
15	9.1	Key indicators with default filters	32
16	9.2	All-India registered cases and annual change	33
17	9.3	Change between first and latest reported year	33
18	B.1	Dataset records	54

CHAPTER - 2

INTRODUCTION

2. INTRODUCTION

2.1 BACKGROUND

Data about the society we live in is produced at a scale that no reader can absorb in raw form. Government agencies, research institutes and the media publish statistics every year, yet most citizens, students and even analysts meet these numbers as long tables inside PDF reports or as a single chart in a news article. Data visualization closes this gap by turning numbers into shapes, positions and colours that the human eye can compare quickly.

One important example is the data on offences against children. The Protection of Children from Sexual Offences (POCSO) Act, 2012 is a special law of India that protects persons below eighteen years of age from sexual assault, sexual harassment and pornography, and lays down child-friendly procedures for reporting, evidence and trial. The National Crime Records Bureau (NCRB), under the Ministry of Home Affairs, publishes the number of cases registered under this Act in its annual *Crime in India* reports, and these figures are often quoted by newspapers and fact-checking organisations.

Python has become the most widely used language for turning such data into insight because of its mature ecosystem: pandas for tabular data, NumPy and SciPy for numerical work, Matplotlib, Seaborn and Plotly for charts. Streamlit adds the last missing piece: it converts an ordinary Python script into an interactive web application, without requiring HTML, CSS or JavaScript knowledge. This project combines these tools into a single interactive and animated dashboard built on one verified dataset of POCSO case counts.

2.2 MOTIVATION

The motivation for this project comes from three observations.

- **Public data is hard to explore.** The case-count figures exist only as scattered values: all-India totals for some years, a few states for others. A reader who wants to know how the numbers changed over time, or how states compare, must search several sources and do the arithmetic manually.
- **Static charts hide structure.** A fixed bar chart cannot be filtered by year or state, and cannot show how the picture changes from one year to the next. Interactive controls and animation let the reader ask their own questions.
- **Sensitive topics need careful presentation.** A dashboard about child abuse must be accurate, must label what is reported and what is derived, and must warn the reader about what the data cannot say. Building such a tool is a good exercise in responsible data communication, not only in programming.

2.3 PROBLEM STATEMENT

To design and develop an interactive, animated, web-based dashboard in Python that explores a single verified dataset of child sexual offence (POCSO) case counts in India, allowing users to filter the data, view it through a wide range of chart types, observe changes over time through animation, and read automatically computed key indicators, while clearly separating reported values from derived values and stating the limitations of the data.

The problem can be divided into four sub-problems:

1. **Data handling:** load a small and irregular dataset (different geographies and years, one row of percentages, several empty columns) and prepare it reliably.
2. **Analysis:** derive meaningful measures (growth, compound annual growth rate, shares, trend) without inventing data.
3. **Visualization:** choose chart types that suit the data and make them interactive and animated.
4. **Interface:** organise everything into a clear dashboard with sidebar filters and no file-upload dependency.

2.4 OBJECTIVES OF THE PROJECT

The main objective is to build a general, user-friendly, filterable dashboard for POCSO case data using only the Python ecosystem. The specific objectives are:

1. To build an interactive Streamlit application for one bundled dataset:
    - The CSV is read from the project folder; no upload control is provided.
    - The application reruns automatically whenever a filter changes.
2. To provide sidebar filters:
    - Year, geography and offence category (multi-select).
    - A range slider on registered case counts and a one-click reset.
3. To implement the requested chart types:
    - Bar, line, pie, donut, radar, histogram, sunburst, time series and scatter plots.
    - A seaborn heatmap as an additional statistical view.
4. To add animation:
    - Play/pause animations for bar, line and scatter charts with an adjustable frame duration.
    - Animated key-indicator cards that count up when the page loads.
5. To apply numerical and statistical methods:
    - Growth, CAGR and shares using NumPy and pandas.
    - Linear trend, kernel density estimate and skewness using SciPy.
6. To communicate the limits of the data honestly: derived values are labelled, empty fields are listed, and the difference between registered cases and actual incidence is stated.
7. To keep the system open-source, lightweight and runnable on an ordinary laptop.

2.5 SCOPE OF THE PROJECT

In scope

- One bundled CSV file with 12 records, covering 2020 to 2023 (details in Chapter 6).
- Descriptive visual analysis: totals, trends, composition, distribution and comparison.
- A single-user, locally hosted Streamlit web application that runs in any modern browser.
- Documentation: source code, dataset description and a user-oriented README.

Out of scope

- Prediction or forecasting. With three national data points, any forecast would be unreliable, so only a descriptive trend line is shown.
- Data for victims by age, conviction rates or investigation outcomes; those columns exist in the dataset but are empty.
- Live data feeds, user accounts, databases and cloud deployment.
- Any identification of individuals; the dataset contains only aggregate counts.

CHAPTER - 3

LITERATURE SURVEY / EXISTING SYSTEM

3. LITERATURE SURVEY / EXISTING SYSTEM

3.1 EXISTING DATA VISUALIZATION TOOLS

Several categories of tools are used today to visualise data like that handled in this project. They range from spreadsheets to professional business-intelligence (BI) platforms and code-based libraries. Table 3.1 summarises the most common ones.

Table 3.1 : Common data visualization tools

Tool	Type	Strengths	Limitations for this use case
Microsoft Excel / Google Sheets	Spreadsheet	Familiar; quick static charts; pivot tables	Limited interactivity; no animation by default; manual rework for every new view
Matplotlib / Seaborn scripts	Python library	Publication-quality static plots; full control	Charts are images; user cannot filter or hover without extra code
Jupyter + ipywidgets	Notebook widgets	Interactive exploration inside a notebook	Needs a running notebook server; not a polished stand-alone application
Tableau, Power BI	Commercial BI	Drag-and-drop dashboards; rich connectors	Proprietary products; limited code-level customisation; not Python-native
Plotly Dash	Python web framework	Highly customisable web dashboards	Callback-based programming model has a steeper learning curve
Streamlit (selected)	Python web framework	Script-to-app model; widgets and caching built in; works with Plotly, Matplotlib	Whole script re-executes on interaction (managed here with caching)

Apart from tools, research on visual communication guides how dashboards should be designed. Table 3.2 lists the works that influenced the design choices in this project.

Table 3.2 : Related literature and its influence on this project

S.No	Reference	Focus	Influence on the project
1	E. R. Tufte, *The Visual Display of Quantitative Information* (2001)	Principles of graphical excellence; showing data with minimal clutter	Plain backgrounds, direct value labels, no decorative 3-D effects
2	W. S. Cleveland and R. McGill, *Graphical Perception*, JASA (1984)	Experiments on how accurately people decode different visual encodings	Bars and positions are used for comparison; pie and donut charts are limited to composition with labels
3	S. Few, *Information Dashboard Design* (2013)	Dashboards should deliver key information at a glance	A row of key-indicator cards above the tabs
4	J. Heer and B. Shneiderman, *Interactive Dynamics for Visual Analysis*, CACM (2012)	Taxonomy of interactive operations such as select, filter and navigate	Sidebar multi-select filters, year selectors, hover details and reset
5	G. Robertson et al., *Effectiveness of Animation in Trend Visualization*, IEEE TVCG (2008)	Compared animated and static trend displays; animation was engaging but less effective for detailed analysis	Animated views are paired with static views, and the animation speed is user-controlled
6	T. Munzner, *Visualization Analysis and Design* (2014)	Framework for choosing visual idioms by task and data type	Each chart type is matched to a question (trend, composition, distribution, comparison)

3.2 LIMITATIONS OF EXISTING SYSTEM

Combining the tools in Table 3.1 with the nature of the POCSO case data, the following limitations are observed.

1. Static presentation:
    - Figures are published as tables or fixed images, so the reader cannot filter by year or state.
    - Any change in view requires recreating the chart manually.
2. Fragmented data:
    - National and state figures are scattered across reports and articles and are not consistently available for every year.
    - Readers must combine and recompute values themselves.
3. Cost and lock-in:
    - Commercial BI platforms are proprietary products and are difficult to integrate into code-driven workflows.
    - They offer less insight into how a visualization is built, which limits their educational value.
4. Weak communication of uncertainty:
    - Charts rarely distinguish reported values from derived values or show which values are lower bounds.
    - Registered cases are often read as the true number of offences.
5. Little support for change over time:
    - Trends are shown as one chart; there is no easy way to step through years and see how composition changes.
6. Barrier for non-programmers:
    - Code-only plots require programming skills, while notebooks require a running kernel.

3.3 PROPOSED SYSTEM

The proposed system is an interactive and animated dashboard written entirely in Python and served through Streamlit. It reads a single bundled CSV file, prepares and derives the data with pandas and NumPy, performs simple statistical analysis with SciPy, and draws the charts with Plotly, Seaborn and Matplotlib. Users interact with it only through the browser: they choose filters in the sidebar, switch between five tabs and play the animations.

The main components of the proposed system are:

- **Data module** – loads the CSV once (cached), converts types, and adds derived columns.
- **Analytics module** – computes key indicators, composition tables, change tables and trend statistics.
- **Visualization module** – a set of figure-building functions, one per chart.

- **Interface module** – sidebar filters, key-indicator cards, tabs, download button and notes.

Table 3.3 : Comparison of existing approaches and the proposed system

Feature	Static scripts / spreadsheets	Commercial BI tools	Proposed system
Interactivity	None or minimal	High	High (filters, hover, play/pause)
Animation	Not available by default	Limited	Bar, line and scatter animation with speed control
Cost	Free / licensed office suite	Licensed	Free and open-source
Python-native and customisable	Partly	No	Yes
Shows derived vs reported values	Rarely	Rarely	Yes, labelled in charts and captions
Data stays on the user's computer	Yes	Depends on setup	Yes (no external services are called)
Educational value (readable source code)	Medium	Low	High

CHAPTER - 4

TECHNOLOGIES AND TOOLS USED

4. TECHNOLOGIES AND TOOLS USED

4.1 PYTHON

Python is a high-level, general-purpose programming language known for readable syntax and a very large collection of scientific and data-analysis libraries. It is the language of the entire project: the data loading, the calculations, the charts and the web interface are all written in one Python file. Python 3.9 or later is recommended.

4.2 STREAMLIT

Streamlit is an open-source Python framework for building data applications. A Streamlit app is an ordinary script; each widget call such as `st.multiselect` both draws the control and returns its current value. When the user changes a widget, Streamlit reruns the script from top to bottom, so the screen always reflects the current inputs. The following Streamlit features are used in this project:

- `st.set_page_config` – wide layout and page title.
- `st.sidebar`, `st.multiselect`, `st.slider`, `st.button` – the filter panel.
- `st.session_state` – stores filter values; a callback resets them.
- `st.cache_data` – loads and parses the CSV only once.
- `st.tabs`, `st.columns` – the page layout.
- `st.plotly_chart`, `st.pyplot`, `st.dataframe` – displaying charts and tables.
- `st.download_button` – exporting the filtered records as CSV.
- `components.html` – the animated key-indicator cards (HTML, CSS and JavaScript).

4.3 PANDAS

pandas provides the DataFrame, a table-like structure with fast filtering, grouping and pivoting. In this project pandas reads the CSV, converts types, creates derived columns (such as geography scope and a date), applies the sidebar filters through Boolean masks, builds pivot tables for the heatmap and computes year-on-year change with `pct_change()`.

4.4 NUMPY

NumPy supplies fast numerical arrays and mathematical functions. It is used for conditional column creation (`np.where`), histogram binning (`np.histogram`), logarithmic axis limits (`np.log10`) and basic statistics such as the median. SciPy builds on NumPy and adds `stats.linregress` for the trend line, `stats.gaussian_kde` for the density curve and `stats.skew` for distribution shape.

4.5 MATPLOTLIB / SEABORN / PLOTLY

Matplotlib is the foundational Python plotting library; it draws the static figure shown in the dashboard and is the engine behind Seaborn. **Seaborn** offers statistical graphics with attractive defaults; here it draws the annotated state-by-year heatmap. **Plotly** produces interactive, browser-based charts with hover, zoom and, importantly, built-in animation frames; it draws most charts of the dashboard.

Table 4.1 : Libraries and their role in the project

Library	Used for	Charts / features
Streamlit	Web interface, widgets, caching	Sidebar, tabs, layout, download
pandas	Data loading, filtering, grouping	All charts and tables
NumPy	Numerical operations	Histogram bins, log scales, medians
SciPy	Statistics	Trend line, KDE curve, skewness
Plotly	Interactive and animated charts	Bar, line, pie, donut, sunburst, radar, scatter, time series, histogram
Seaborn / Matplotlib	Statistical static graphics	State × year heatmap

CHAPTER - 5

SYSTEM REQUIREMENTS

5. SYSTEM REQUIREMENTS

5.1 HARDWARE REQUIREMENTS

The dashboard processes only twelve records, so its hardware needs are modest. The figures below are recommended configurations for a smooth experience with the animations; they are not measured limits.

Table 5.1 : Recommended hardware configuration

Component	Minimum	Recommended
Processor	Dual-core, 2 GHz (Intel Core i3 or equivalent)	Quad-core (Intel Core i5 / AMD Ryzen 5 or better)
Memory (RAM)	4 GB	8 GB
Storage	1 GB free (Python environment and packages)	2 GB free
Display	1366 × 768	1920 × 1080 (wide layout)
Network	Needed only to install packages	Not needed at run time

5.2 SOFTWARE REQUIREMENTS

Table 5.2 : Software configuration

Item	Specification
Operating system	Windows 10/11, Ubuntu / other Linux, or macOS
Language	Python 3.9 or later
Framework	Streamlit 1.35 or later
Libraries	pandas ≥ 2.0, NumPy ≥ 1.24, SciPy ≥ 1.10, Matplotlib ≥ 3.7, Seaborn ≥ 0.13, Plotly ≥ 5.20
Browser	Current Chrome, Edge, Firefox or Safari (JavaScript enabled)
Editor	Visual Studio Code, PyCharm or any text editor

Installation and execution use two commands, run from the project folder:

```
pip install -r requirements.txt
streamlit run app.py
```

CHAPTER - 6

DATASET DESCRIPTION

6. DATASET DESCRIPTION

6.1 DATASET SOURCE

The only data used in this project is the file `child_sexual_offence_cases_india_verified.csv`. It contains 12 rows. The values were compiled from the NCRB *Crime in India* reports for the years 2020 to 2023 as reproduced by the secondary sources named in the file's `source` column (for example Deccan Herald, Factly and NewsClick). The dashboard does not download any additional data and has no file-upload option, so every figure on screen can be traced back to this one file.

6.2 DATASET ATTRIBUTES

Table 6.1 : Attributes of the dataset

Attribute	Description	Populated rows
year	Calendar year of the report (2020–2023)	12 of 12
geography	“India (all)” or the name of a state	12 of 12
offence_category	POCSO offence group, e.g. all offences, Sec 4 & 6, or a victim-share description	12 of 12
total_cases_registered	Number of registered cases	11 of 12
victims_girls, victims_boys, victims_other_gender	Victim counts by gender	0 of 12 (empty)
cases_solved_chargesheeted	Cases with charge-sheet filed	0 of 12 (empty)
cases_convicted	Cases ending in conviction	0 of 12 (empty)
source	Publication from which the value was taken	12 of 12
notes	Remarks such as “reported as more than 28,000”	2 of 12

6.3 DATA TYPES

Table 6.2 : Data types after loading

Attribute	Type in pandas	Type in dashboard use
year	int64	Integer; also converted to a year-end date for the time series
geography, offence_category, source, notes	object (text)	Categorical filters and labels
total_cases_registered	float64 (one missing value)	Numeric measure (alias *cases*)
Empty numeric columns	float64 (all missing)	Detected automatically and listed, not plotted

6.4 DATA PREPROCESSING

Preprocessing is performed in the function `load_data()`, which is cached so that it runs only once per session. The following steps are applied:

1. **Numeric conversion** – `total_cases_registered` is converted to numbers and copied to a short column *cases*.
2. **Short offence names** – the long category text is mapped to "All POCSO offences", "Sec 4 & 6 (penetrative)" or "Victim gender share".
3. **Scope column** – rows whose geography starts with "India" are marked National; all others are State.
4. **Date column** – each year is turned into a year-end date (31 December) for use on a time axis.

The dataset's coverage after preprocessing is shown in Table 6.4.

Table 6.3 : Coverage of case-count records

Geography	2020	2021	2022	2023
India (all) – all offences	47,221	53,874	63,414	–
India (all) – Sec 4 & 6	> 28,000 (floor)	–	–	–
Uttar Pradesh	–	7,129	8,136	–
Maharashtra	–	6,200	–	–
Madhya Pradesh	–	6,070	–	–
Tamil Nadu	–	4,465	–	–
Karnataka	–	2,813	–	3,878

6.5 DATA CLEANING

The dataset is small but irregular, so cleaning is concerned with correctness and honesty rather than with large-scale repair.

- **Missing case count.** One row (the victim gender share) has no case count. It is separated from the count records and used only for the donut chart. The values are parsed from its note with a regular expression: girls 97.1%–97.6% and boys 2.3%–2.8%. The donut uses the midpoint of each range, and the small remainder is shown as "unspecified / rounding".
- **Empty columns.** Columns that contain no value at all are detected programmatically, listed in the Data & notes tab and excluded from every chart; nothing is estimated to fill them.
- **Lower bound.** The 2020 Sec 4 & 6 value is reported only as "more than 28,000". It is treated as a lower bound and labelled accordingly.
- **Derived components.** "Rest of India" (national total minus the listed states) and "Other POCSO offences" (2020 total minus Sec 4 & 6) are calculated only where both quantities exist, and they are labelled as derived.
- **No imputation.** Missing state-years are shown as blanks (grey cells in the heatmap, gaps in the radar), never filled.

Table 6.4 : Descriptive statistics of the case-count records

Subset	n	Mean	Median	Std. dev.	Min	Max	Skewness
All records	11	21,018	7,129	23,051	2,813	63,414	0.88

Subset	n	Mean	Median	Std. dev.	Min	Max	Skewness
State-level only	7	5,527	6,070	1,886	2,813	8,136	-0.09

CHAPTER - 7

IMPLEMENTATION

7. IMPLEMENTATION

7.1 PROJECT STRUCTURE

The application is deliberately compact. All logic lives in one Python file, and the data file sits beside it.

```
project/
├── app.py                                          # Streamlit application
├── child_sexual_offence_cases_india_verified.csv   # dataset (read-only, bundled)
├── requirements.txt                                # dependencies
└── README.md                                       # usage notes
```

Table 7.1 : Project files

File	Purpose
app.py	Data layer, analytics, all chart functions, and the Streamlit interface
child_sexual_offence_cases_india_verified.csv	The only data source
requirements.txt	Minimum versions of Streamlit, pandas, NumPy, SciPy, Matplotlib, Seaborn and Plotly
README.md	Installation, features, derived values and troubleshooting

Figure 7.1 shows the layered architecture. The data layer feeds the processing layer; the visualization layer draws charts from the filtered data; and the presentation layer, which is Streamlit, arranges everything for the user.

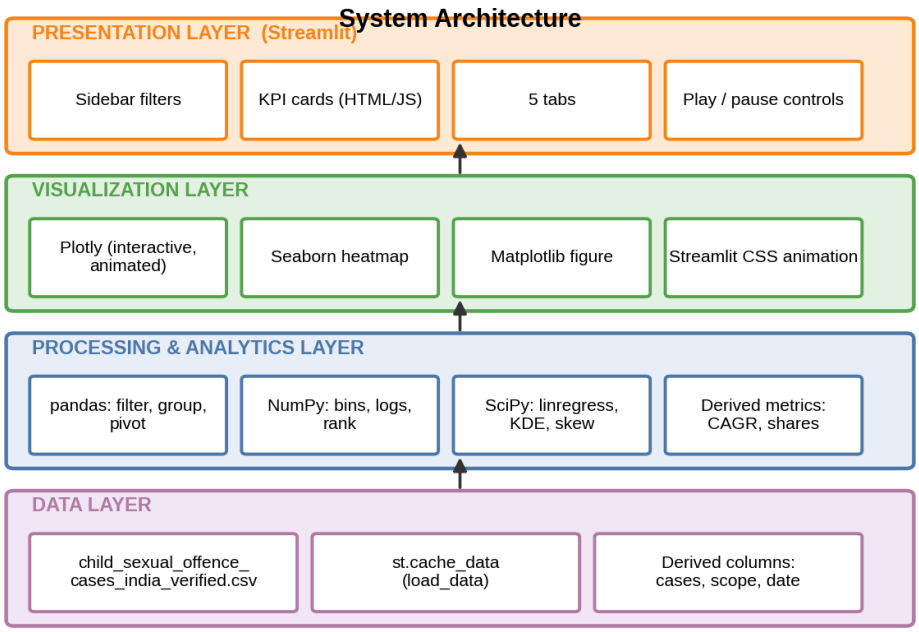


Figure 7.1 : System architecture of the dashboard

7.2 DATA LOADING

The CSV path is resolved relative to `app.py`, so the application runs from any working directory. The loader is decorated with `@st.cache_data`; Streamlit therefore reads and prepares the file only on the first run, even though the script re-executes on every interaction. If the file is missing, the app shows an error message and stops.

```
def load_data() -> pd.DataFrame:
    raw = pd.read_csv(DATA_PATH)
    raw["cases"] = pd.to_numeric(raw["total_cases_registered"], errors="coerce")
    raw["offence_short"] = raw["offence_category"].map(short_offence)
    raw["scope"] = np.where(raw["geography"].str.startswith("India"), "National", "State")
    raw["date"] = pd.to_datetime(raw["year"].astype(str) + "-12-31")
    return raw
```

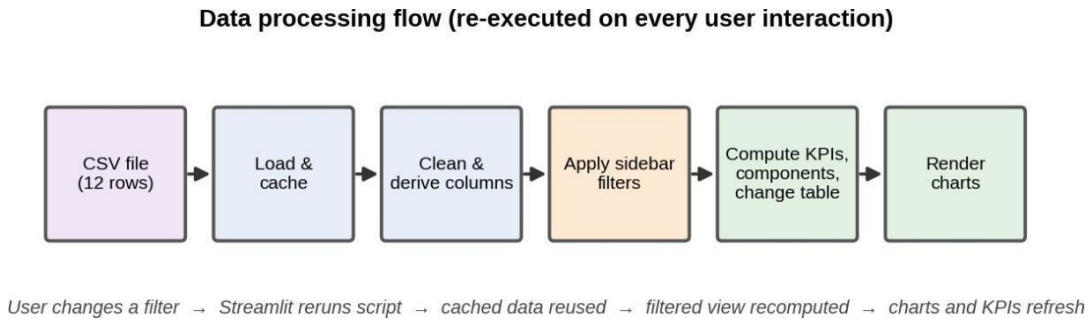


Figure 7.2 : Data processing flow

7.3 DATA PREPROCESSING

After loading, three helper functions prepare the data for analysis. `short_offence()` shortens category names. `build_components()` splits each national total into parts that add back up to it, which feeds the pie and sunburst charts. `change_table()` compares the first and latest reported year of every series with at least two years, computing the percentage change and the compound annual growth rate (CAGR).

```
def change_table(f: pd.DataFrame) -> pd.DataFrame:
    """First-vs-latest reported year for every series with >= 2 reported years."""
    d = f[f["offence_short"] == ALL_OFF].sort_values("year")
    g = d.groupby("geography").agg(
        first_year=("year", "first"), last_year=("year", "last"),
        first_cases=("cases", "first"), last_cases=("cases", "last"), n=("year", "size"),
    )
    g = g[g["n"] >= 2].reset_index()
    if g.empty:
        return g
    span = (g["last_year"] - g["first_year"]).astype(float)
    g["change_pct"] = (g["last_cases"] / g["first_cases"] - 1) * 100
    g["cagr_pct"] = ((g["last_cases"] / g["first_cases"]) ** (1 / span) - 1) * 100
    return g
```

The CAGR is calculated as (latest / first)$^{1/\text{years}}$ − 1. For example, Karnataka rose from 2,813 (2021) to 3,878 (2023), a change of +37.9% or 17.4% per year.

7.4 VISUALIZATION DEVELOPMENT

Every chart is produced by its own function that receives the filtered DataFrame and returns a figure object. This keeps chart logic separate from page layout and makes each function easy to test and reuse. Plotly animation is based on *frames*: one frame per year for the bar and scatter charts, and cumulative frames for the line chart so that the lines appear to be drawn year by year.

Two small helpers apply to all animated charts. `update_all_traces()` applies a style to the figure and to every frame, because Plotly's normal update skips frames. `play_controls()` adds the Play / Pause buttons and the year slider, taking the frame duration from the sidebar so that one setting controls every animation. A complete example of a chart function is the animated bar chart:

```
def fig_bar_race(f, mode, speed, off_colors):
    d = f.copy()
    if mode == "States only":
        d = d[d["scope"] == "State"]
    elif mode == "National only":
        d = d[d["scope"] == "National"]
    if d.empty:
        return None
    log = mode.startswith("All")
    years = sorted(int(y) for y in d["year"].unique())
    order = [g for g in GEO_ORDER if g in set(d["geography"])]
    fig = px.bar(
        d, x="geography", y="cases", color="offence_short", animation_frame="year",
        barmode="group", text_auto=",.0f",
        category_orders={"year": years, "geography": order},
        color_discrete_map=off_colors,
        labels={"cases": "Registered cases", "geography": "", "offence_short": "Offence category"},
    )
    update_all_traces(fig, textposition="outside", cliponaxis=False)
    fig.update_yaxes(**axis_range(d["cases"].min(), d["cases"].max(), log))
    base_layout(fig, "Registered cases by geography - press \u25b6 Play to step through the years",
      520)
    return play_controls(fig, [fr.name for fr in fig.frames], speed)
```

7.5 STREAMLIT INTERFACE DEVELOPMENT

The interface follows the order of the script. The page is configured first (wide layout, title). A small block of CSS adds a fade-in and slide-in effect to charts and the title. The sidebar is then built, the data are filtered, and the main area is drawn: title, key-indicator cards, and five tabs.

The key-indicator cards are rendered through `components.html`, because a short JavaScript is needed to count each number up from zero when the page loads. The numbers use Indian digit grouping. If the embed fails, the code falls back to plain `st.metric` cards. To remain compatible with different Streamlit versions, a helper (`show()`) selects the correct width argument for `st.plotly_chart` at run time.

7.6 INTERACTIVE FILTERS

All filters live in the sidebar and are stored in `st.session_state` under fixed keys. The filtered DataFrame is produced by combining the conditions with a single Boolean mask. The file-upload option was intentionally left out.

Table 7.2 : Sidebar controls

Control	Type	Effect
Year	Multi-select	Keeps only the chosen years
Geography	Multi-select	Keeps India (all) and/or selected states
Offence category	Multi-select	Keeps all-offence and/or Sec 4 & 6 records
Registered cases (range)	Range slider	Keeps records within the chosen minimum and maximum
Reset filters	Button with callback	Restores every filter to its default
Frame duration (ms)	Slider	Controls the speed of all animated charts

```
mask = (cases["year"].isin(st.session_state["f_years"])
        & cases["geography"].isin(st.session_state["f_geos"])
        & cases["offence_short"].isin(st.session_state["f_offs"])
        & cases["cases"].between(*st.session_state["f_range"]))
f = cases[mask].sort_values(["year", "geography"]).copy()
```

If the filters leave no records, the dashboard displays a warning and stops. Individual charts also check their own minimum requirements (for example, a radar chart needs three or more states) and show a short message instead of an error.

7.7 DASHBOARD INTEGRATION

The final dashboard integrates all components in five tabs. Every tab reads the same filtered data, so a change in the sidebar updates all charts consistently.

Table 7.3 : Dashboard tabs and the charts they contain

Tab	Contents
Overview	Animated bar chart (with scope selector and log-scale option), donut, pie with year selector, sunburst
Trends	Animated line chart, time series with range slider and trend line, change table
Geography	Radar chart of states by year; seaborn heatmap (state × year)
Distribution & relationships	Histogram with density curve and statistics; animated rank scatter; first-vs-latest scatter
Data & notes	Filtered table, CSV download, dataset notes and sources

CHAPTER - 8

DATA VISUALIZATION

8. DATA VISUALIZATION

This chapter describes each chart type, the question it answers, how it was implemented and how the user can interact with it. The figures in this report are static renderings of the same data produced with Matplotlib and Seaborn; in the running application, most of them are interactive Plotly charts.

Table 8.1 : Summary of chart types

Chart	Library	Question answered	Interaction
Bar (animated)	Plotly	How do geographies compare in each year?	Play/pause, year slider, scope, log scale
Line (animated)	Plotly	How did each series change over time?	Play/pause, log scale, hover
Pie / Donut	Plotly	What share does each part contribute?	Year selector, hover
Scatter (2 types)	Plotly	How do states rank? Did values rise or fall?	Animation, hover
Histogram	Plotly + SciPy	How are case counts distributed?	Number of bins, record scope
Sunburst	Plotly	How do years split into components?	Click to drill down
Radar	Plotly	What is the profile of states in a year?	Hover, legend toggle
Time series	Plotly + SciPy	What is the trend and annual change?	Series selector, range slider
Heatmap	Seaborn	Which state-years have the highest values?	Static (filtered)

8.1 BAR CHART

- **Purpose:** compare values across geographies and years. Bars use position along a common baseline, which people judge more accurately than angle or area.
- **Data:** registered cases per geography, grouped by offence category, one animation frame per year.
- **Implementation:** `px.bar` with `animation_frame='year'`; fixed axis ranges so bars do not rescale between frames.
- **Interaction:** a scope selector (all records on a log scale, states only, national only), Play / Pause, and a year slider.

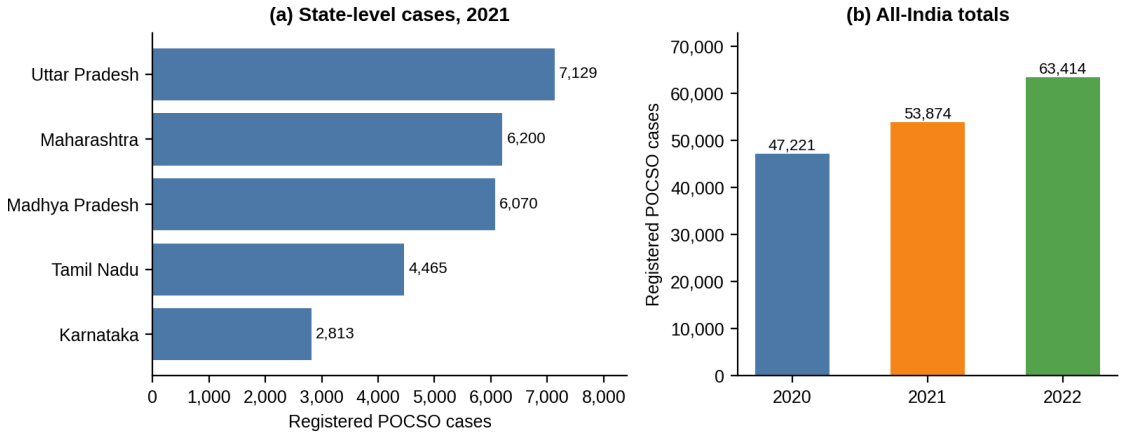


Figure 8.1 : Bar charts: (a) state-level cases in 2021; (b) all-India totals 2020–2022

In 2021 Uttar Pradesh recorded the highest count among the listed states (7,129), followed by Maharashtra (6,200) and Madhya Pradesh (6,070). The all-India bars rise in each year.

8.2 LINE CHART

- **Purpose:** show change over time for every series that has more than one point.
- **Data:** all-offence case counts per geography per year (national 2020–2022; Uttar Pradesh 2021–2022; Karnataka 2021 and 2023).
- **Implementation:** `go.Scatter` traces with cumulative frames so that pressing Play redraws the lines year by year; an optional log scale lets national and state values share one axis.
- **Interaction:** play, pause, slider, legend click to hide a series, hover values.

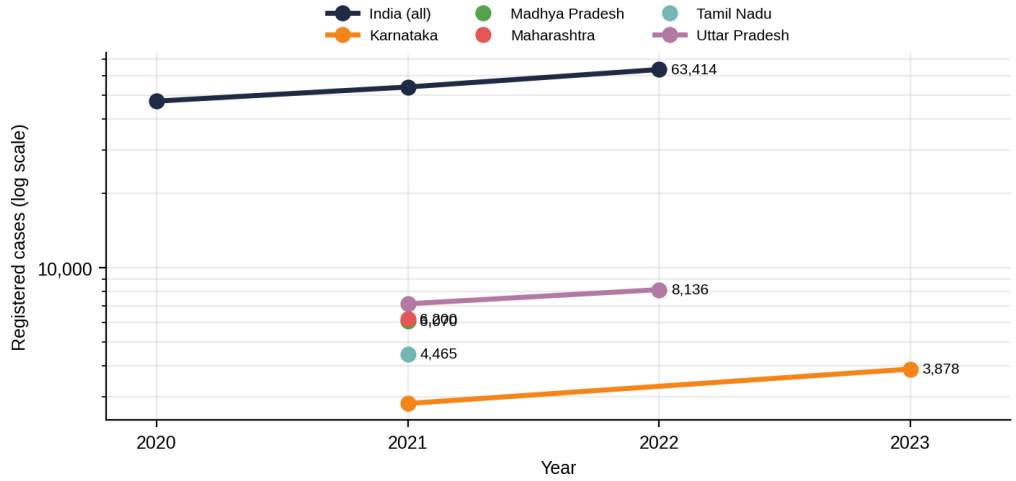


Figure 8.2 : Line chart of registered cases by geography (log scale)

8.3 PIE CHART

- **Purpose:** show how a total divides into parts. Pie charts are used only for a few slices and always with labels.
- **Data:** the 2021 national total divided into five listed states and the derived "Rest of India".
- **Implementation:** `go.Pie` fed by the components table; the year is chosen from a drop-down.
- **Interaction:** year selector and hover details; clicking a legend entry hides a slice.

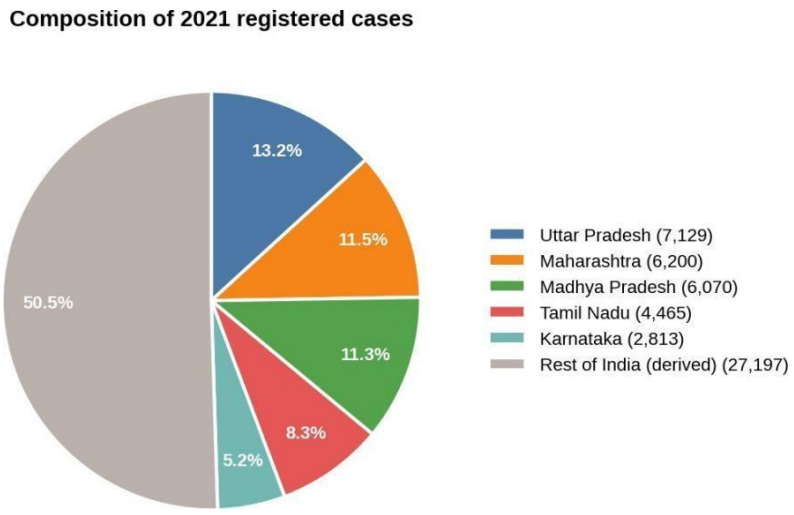


Figure 8.3 : Pie chart: composition of 2021 registered cases

The five listed states together account for 49.5% of the 2021 all-India total (26,677 of 53,874); the remaining 50.5% (27,197) is derived as the rest of India.

8.4 SCATTER PLOT

- **Rank scatter (animated):** each state is a bubble placed by its rank within the year (x) and its cases (y, log scale); bubble size shows cases. Pressing Play steps through the years that have state data.
- **First-vs-latest scatter:** for each series with two or more years, the first reported value (x) is plotted against the latest (y) on log–log axes. Points above the dotted y = x line indicate an increase.

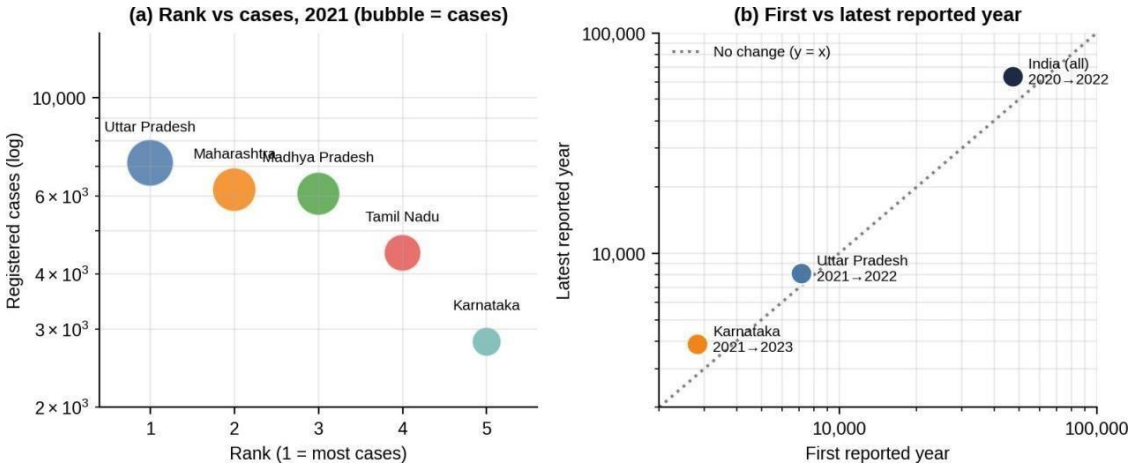


Figure 8.4 : Scatter plots: (a) rank vs cases for 2021; (b) first vs latest reported year

All three series in panel (b) lie above the line of no change, meaning every series with two reported years shows a higher latest value than its first.

8.5 HISTOGRAM

- **Purpose:** display how case counts are distributed across records.
- **Data:** the 7 state-level records (the user may switch to national or all records).
- **Implementation:** `np.histogram` for the bins and `scipy.stats.gaussian_kde` for a smooth density curve scaled to the bar heights; mean, median and skewness are shown beneath the chart.
- **Interaction:** a slider for the number of bins (3 to 12) and a selector for which records to include.

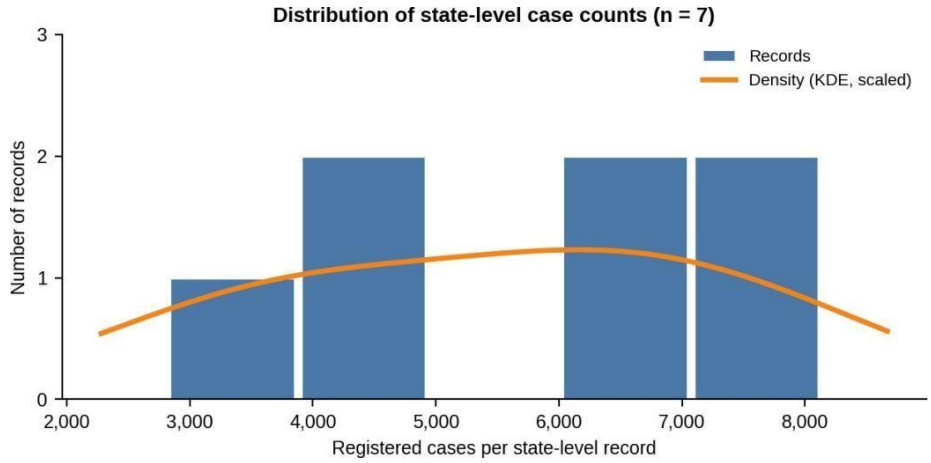


Figure 8.5 : Histogram with scaled kernel density estimate of state-level case counts

With only 7 values, the histogram is descriptive. The state-level mean is 5,527 cases and the median 6,070; the dashboard states this small-sample caution below the chart.

8.6 OTHER VISUALIZATIONS USED

8.6.1 Donut Chart

The donut shows the victim gender split from the single percentage row of the dataset. Girls account for 97.1%–97.6% and boys for 2.3%–2.8%; the chart uses the midpoints (97.35% and 2.55%) and shows the remainder as a thin grey slice. It is a fixed reference chart and is not affected by the filters.

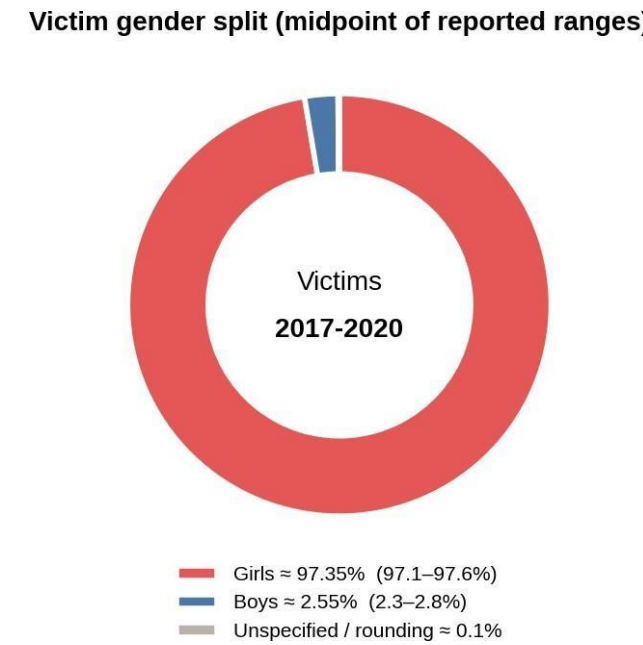


Figure 8.6 : Donut chart: victim gender split, 2017–2020 (percentages, not counts)

8.6.2 Radar Chart

The radar chart has one axis per state and one polygon per year, so several states can be compared at once. Years with values for only some states appear as points because the missing values are not estimated.

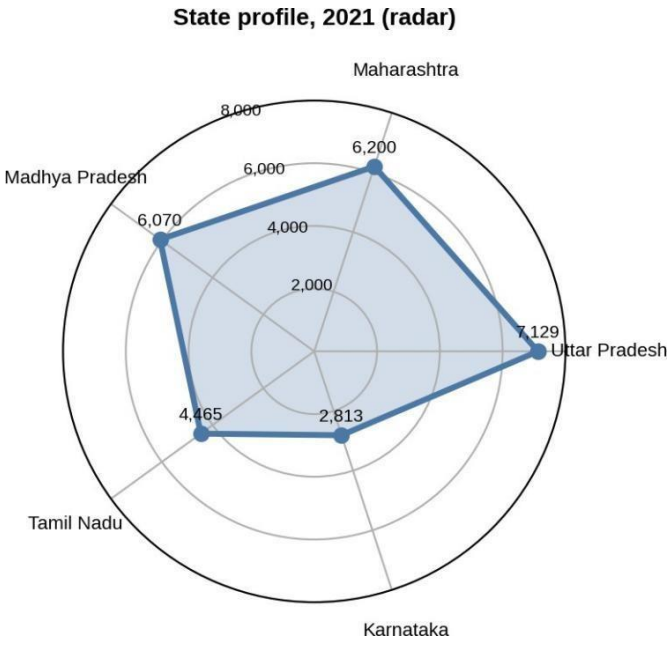


Figure 8.7 : Radar chart: registered cases by state, 2021

8.6.3 Sunburst Chart

The sunburst places years in the inner ring and their components in the outer ring: listed states and the derived “Rest of India” for 2021 and 2022, and Sec 4 & 6 and “Other offences” for 2020. The year 2023 is labelled as partial because only Karnataka is listed. In the application the user can click a ring segment to zoom in.

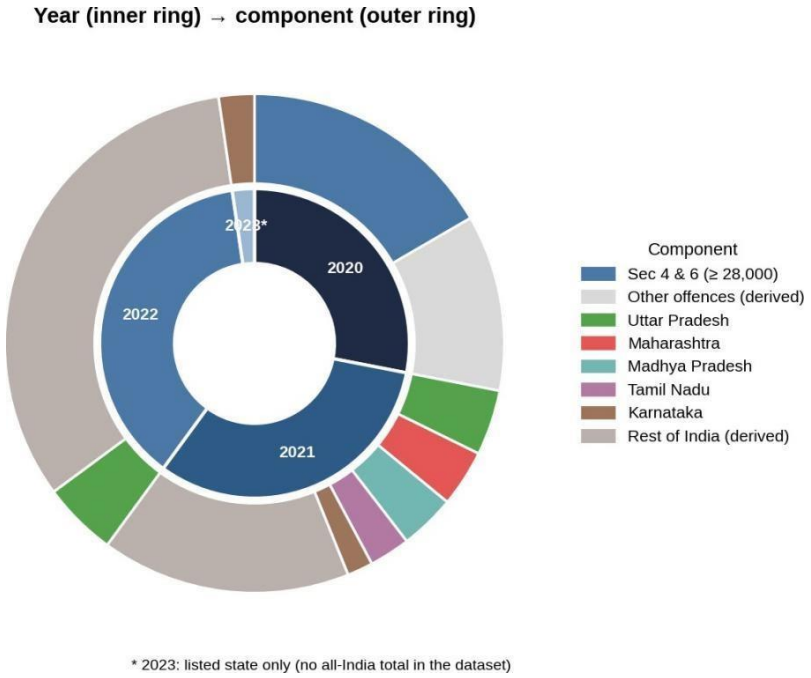


Figure 8.8 : Sunburst chart (shown as nested rings): year → component

8.6.4 Time Series

The time series plots the all-India totals against year-end dates, with bars showing the percentage change from the previous reported year and a dashed linear trend fitted with `scipy.stats.linregress` when a series has at least three years. For the national series the fitted slope is +8,096 cases per year (RZ = 0.99, n = 3). The fit is descriptive only and is not a forecast.

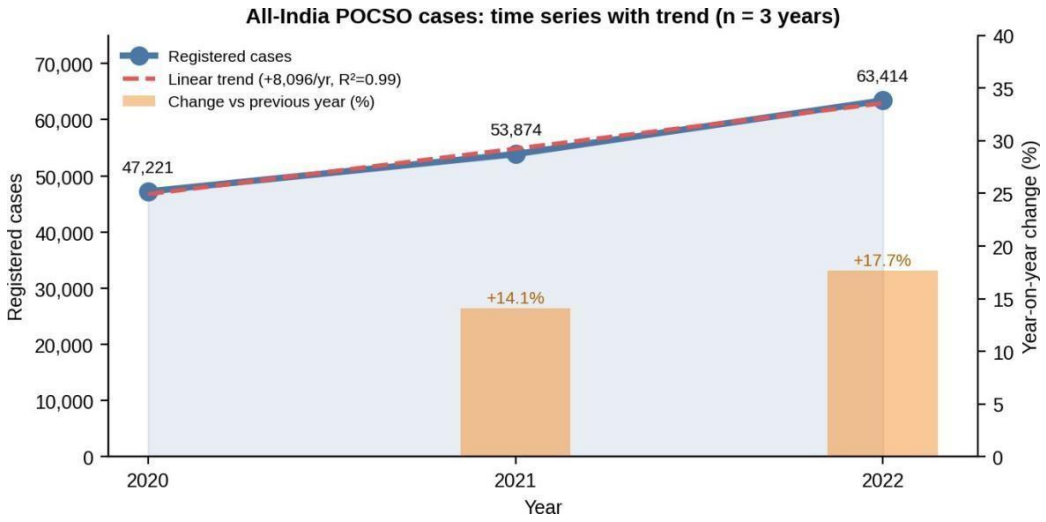


Figure 8.9 : Time series of all-India registered cases with year-on-year change and linear trend

8.6.5 Heatmap

The seaborn heatmap arranges states as rows and years as columns, colouring each cell by case count. Grey cells make the gaps in the dataset visible instead of hiding them.

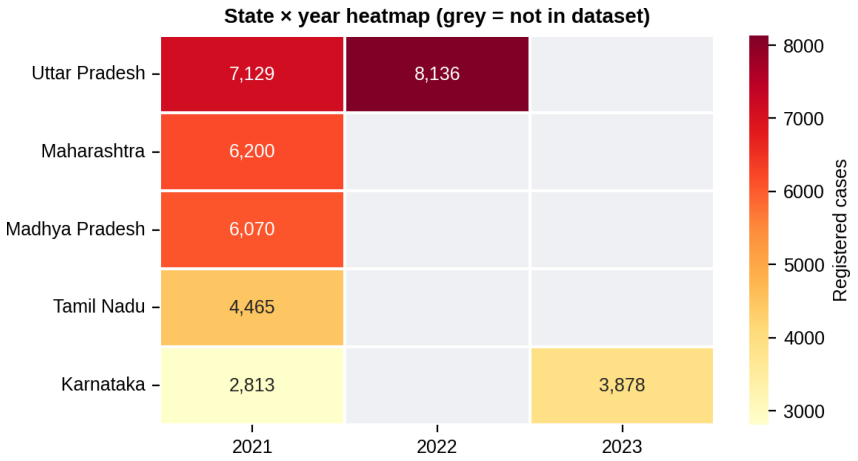


Figure 8.10 : Seaborn heatmap of state-level cases by year

CHAPTER - 9

RESULTS AND DISCUSSION

9. RESULTS AND DISCUSSION

9.1 DASHBOARD SCREENSHOTS

Figure 9.1 is a schematic of the final dashboard. The sidebar on the left holds the filters and the animation speed. The main area shows the title, five key-indicator cards, the five tabs and the charts of the selected tab. Screenshots of the running application for each tab are collected in Appendix C.

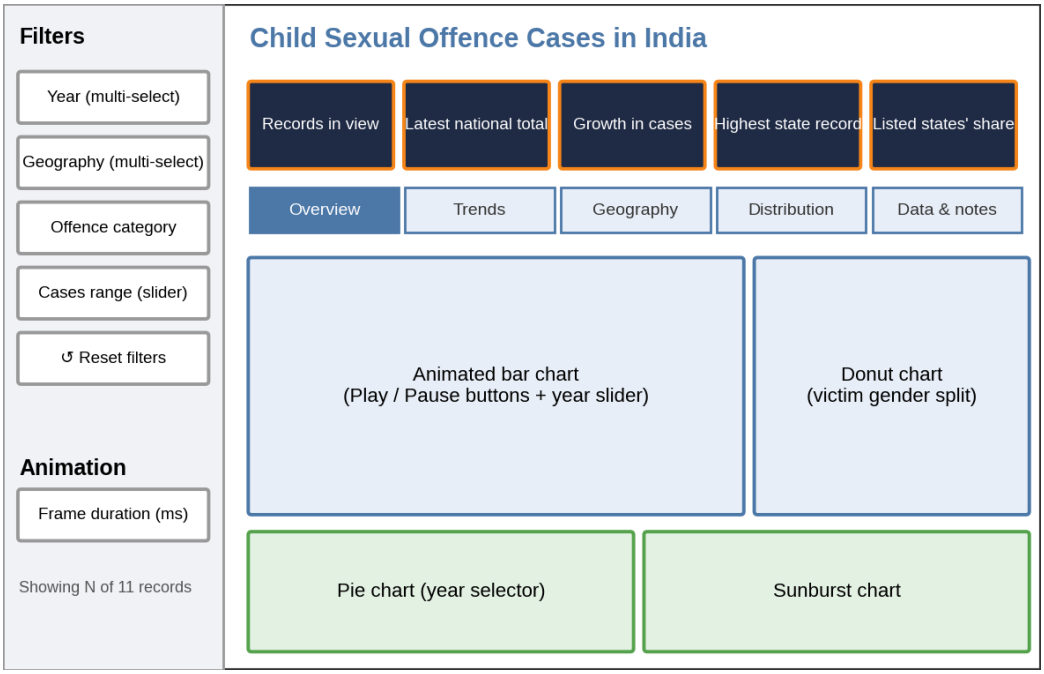


Figure 9.1 : Schematic layout of the dashboard (not a screenshot)

9.2 VISUALIZATION RESULTS

With all filters at their defaults the key-indicator cards show the values in Table 9.1.

Table 9.1 : Key indicators with default filters

Indicator	Value	How it is obtained
Records in view	11	Records that have a case count
Latest national total	63,414 (2022)	All-offence, all-India record of the latest year
Growth in registered cases	+34.3%	2020 → 2022 (47,221 → 63,414); CAGR 15.9% per year
Highest state record	8,136 (Uttar Pradesh, 2022)	Largest state-level value in view
Listed states' share of national	49.5%	Five listed states in 2021 divided by the 2021 national total

Table 9.2 : All-India registered cases and annual change

Year	Registered cases	Change vs previous year
2020	47,221	–
2021	53,874	+14.1%
2022	63,414	+17.7%

Table 9.3 : Change between first and latest reported year

Series	From	To	Cases (from → to)	Change	CAGR / yr
India (all)	2020	2022	47,221 → 63,414	+34.3%	15.9%
Uttar Pradesh	2021	2022	7,129 → 8,136	+14.1%	14.1%
Karnataka	2021	2023	2,813 → 3,878	+37.9%	17.4%

9.3 ANALYSIS OF RESULTS

National trend

Registered POCSO cases in the all-India series rose from 47,221 in 2020 to 53,874 in 2021 (+14.1%) and to 63,414 in 2022 (+17.7%). Over the two years this is an increase of 34.3%, or 15.9% per year when compounded. A straight line fitted through the three points rises by about 8,096 cases per year with RZ = 0.99; with only three points, this indicates direction but has no predictive strength.

Geographical concentration

In 2021 the five listed states contributed 26,677 cases, 49.5% of the national total. Uttar Pradesh alone accounted for 13.2%, and the top three listed states (Uttar Pradesh, Maharashtra and Madhya Pradesh) for 36.0%. The derived remainder of 27,197 cases (50.5%) belongs to states and union territories not listed in the dataset. In 2022 the dataset lists only Uttar Pradesh, with 8,136 cases (12.8% of that year's national total).

Change in individual states

Uttar Pradesh rose by 14.1% between 2021 and 2022, and Karnataka by 37.9% between 2021 and 2023 (17.4% a year). Both are above the line of no change in the scatter plot of Figure 8.4.

Offence composition and victims

For 2020 the dataset reports that more than 28,000 cases fell under Sec 4 & 6 (penetrative assault), which is at least 59.3% of the 47,221 total. Because the figure is a lower bound, the true share may be higher. The gender-share row reports that girls are about 97% of victims and boys about 2.3%–2.8%.

Distribution

The state-level records are spread between 2,813 and 8,136 cases, with mean 5,527, median 6,070 and skewness -0.09. Mixing national and state records produces a strongly two-group distribution, which is why the histogram offers a record-scope selector.

9.4 KEY INSIGHTS

1. Registered cases in the all-India series grew 34.3% between 2020 and 2022, with the year-on-year increase rising from 14.1% to 17.7%.
2. Almost half of the 2021 national total (49.5%) is covered by only five listed states, and Uttar Pradesh is the largest of those five (13.2% of the national total).
3. Every series that has two reported years shows a higher latest value than its first.
4. Girls make up the large majority of reported victims (about 97%).
5. **Interpretation caution:** the figures are registered cases. A rise can reflect better awareness, easier reporting and stronger registration as well as, or instead of, more offences, and differences between states may reflect reporting practices. The dashboard therefore does not rank states as safer or less safe.
6. Coverage gaps matter: the dataset has no state data for 2020, only one state each for 2022 and 2023, and empty columns for victims, charge-sheets and convictions.

CHAPTER - 10

ADVANTAGES AND LIMITATIONS

10. ADVANTAGES AND LIMITATIONS

10.1 ADVANTAGES

1. Interactive and animated:
    - Filters, hover, play/pause, year sliders and count-up indicators make exploration engaging.
2. Wide chart coverage:
    - Bar, line, pie, donut, radar, histogram, sunburst, time series, scatter and heatmap in one application.
3. Honest data presentation:
    - Derived values are labelled; lower bounds and empty fields are disclosed; small-sample cautions are shown.
4. Open-source and low cost:
    - Uses only free Python libraries; runs on an ordinary laptop.
5. Simple to run:
    - Two commands; the data file is bundled, so no upload or configuration is needed.
6. Efficient:
    - Data is loaded once with caching, and charts are built by small reusable functions.
7. Maintainable and educational:
    - A single readable Python file, organised into data, analytics, chart and interface sections.
8. Private:
    - All processing is local and no external service is contacted at run time.

10.2 LIMITATIONS

1. Very small dataset:
    - Only 12 records (11 with counts), so statistics such as the trend line, density and skewness are descriptive, not conclusive.
2. Uneven coverage:
    - National totals exist for 2020–2022 only; state values cover five states in 2021, and one state in each of 2022 and 2023.
3. Empty columns:
    - Victim counts by gender, charge-sheeted cases and convictions are empty, so those aspects cannot be analysed.
4. Registered cases only:
    - The data cannot measure the true prevalence of offences or compare safety between states.
5. Secondary sources:

- Values are compiled from NCRB figures as reported by media and fact-checkers, not extracted directly from the NCRB tables.

6. Static input:
   - The dashboard reads one bundled CSV; updating it requires replacing the file.
7. No forecasting:
   - Prediction is intentionally excluded because of the small number of time points.
8. Script re-execution:
   - Streamlit reruns the script on each interaction; this is acceptable for 12 records but would need optimisation for large data.

CHAPTER - 11

CONCLUSION

11. CONCLUSION

This project set out to build an interactive and animated dashboard that makes a small, irregular and sensitive dataset of POCSO case counts understandable. Using Python, Streamlit, pandas, NumPy, SciPy, Matplotlib, Seaborn and Plotly, a single application was developed that offers sidebar filters, key-indicator cards, and ten kinds of visualization: bar, line, pie, donut, radar, histogram, sunburst, time series, scatter and heatmap. Animations let users step through years, and all controls respond immediately without any file upload.

The analysis shows a rise in registered cases at the national level from 47,221 in 2020 to 63,414 in 2022, geographical concentration in a few large states (five listed states make up 49.5% of the 2021 total), and a victim profile in which girls form the large majority. These results are reported together with their limits: the data are registered cases rather than incidence, coverage is partial, several columns are empty, and derived values are labelled as such.

Beyond the numbers, the project demonstrates a complete data-application workflow: loading and cleaning data, deriving indicators, choosing suitable charts, building an interface and documenting the result. Because the dashboard is modular and open-source, it can be extended with new data, charts and analyses, and it serves as a basis for further study of child-protection data.

CHAPTER - 12

FUTURE ENHANCEMENT

12. FUTURE ENHANCEMENT

The following enhancements can extend the project:

1. Richer and fuller data:
   - Enhancement: add state-wise and year-wise tables directly from NCRB publications, with victim age and gender counts, charge-sheet and conviction data.
   - Benefit: allows pendency, conviction and age-group analysis that is impossible now.
2. Per-capita and rate measures:
   - Enhancement: add child population data and compute rates per 100,000 children.
   - Benefit: fairer comparison between states of different size.
3. Map-based view:
   - Enhancement: add a choropleth map of India with a year slider.
   - Benefit: immediate spatial understanding.
4. Automatic data updates:
   - Enhancement: read new NCRB releases from a maintained file or API.
   - Benefit: the dashboard stays current without code changes.
5. Export and reporting:
   - Enhancement: generate PDF or PowerPoint summaries of the current view.
   - Benefit: easier sharing with stakeholders.
6. Cloud deployment:
   - Enhancement: publish the app on Streamlit Community Cloud or a college server.
   - Benefit: access from any device without local installation.
7. Multilingual and mobile-friendly interface:
   - Enhancement: Hindi, Telugu and other language labels; responsive layout.
   - Benefit: wider reach.
8. Automated testing:
   - Enhancement: add unit tests for the data and analytics functions and UI tests for the dashboard.
   - Benefit: safer future changes.

13. REFERENCES

[1] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007, doi: 10.1109/MCSE.2007.55.

[2] M. Waskom, "seaborn: statistical data visualization," *Journal of Open Source Software*, vol. 6, no. 60, p. 3021, 2021, doi: 10.21105/joss.03021.

[3] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf.*, Austin, TX, USA, 2010, pp. 51–56.

[4] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020, doi: 10.1038/s41586-020-2649-2.

[5] P. Virtanen et al., "SciPy 1.0: fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[6] Plotly Technologies Inc., *Collaborative data science*, Montréal, QC, Canada, 2015. [Online]. Available: https://plot.ly

[7] Streamlit Inc., "Streamlit documentation." [Online]. Available: https://docs.streamlit.io

[8] J. VanderPlas, *Python Data Science Handbook: Essential Tools for Working with Data*, 1st ed. Sebastopol, CA, USA: O'Reilly Media, 2016.

[9] E. R. Tufte, *The Visual Display of Quantitative Information*, 2nd ed. Cheshire, CT, USA: Graphics Press, 2001.

[10] W. S. Cleveland and R. McGill, "Graphical perception: Theory, experimentation, and application to the development of graphical methods," *Journal of the American Statistical Association*, vol. 79, no. 387, pp. 531–554, 1984.

[11] S. Few, *Information Dashboard Design: Displaying Data for At-a-Glance Monitoring*, 2nd ed. Burlingame, CA, USA: Analytics Press, 2013.

[12] J. Heer and B. Shneiderman, "Interactive dynamics for visual analysis," *Communications of the ACM*, vol. 55, no. 4, pp. 45–54, 2012.

[13] G. Robertson, R. Fernandez, D. Fisher, B. Lee and J. Stasko, "Effectiveness of animation in trend visualization," *IEEE Transactions on Visualization and Computer Graphics*, vol. 14, no. 6, pp. 1325–1332, 2008.

[14] T. Munzner, *Visualization Analysis and Design*. Boca Raton, FL, USA: CRC Press, 2014.

[15] Government of India, Ministry of Law and Justice, *The Protection of Children from Sexual Offences Act, 2012* (Act No. 32 of 2012).

[16] National Crime Records Bureau, Ministry of Home Affairs, Government of India, *Crime in India* (annual reports, 2020–2023). [Online]. Available: https://ncrb.gov.in

[17] Secondary reports named in the dataset's *source* column (Deccan Herald, Factly, NewsClick and others) for the compiled figures.

CHAPTER - 14

APPENDIX

14. APPENDIX

A SOURCE CODE

Complete source code of `app.py` (line numbers added for reference). Emoji and other pictographic characters that the listing font cannot print are shown as Python \uXXXX escapes.

```
"""
Child Sexual Offence (POCSO) Cases in India - animated Streamlit dashboard
==========================================================================
Data source : child_sexual_offence_cases_india_verified.csv (bundled, no upload)
Run         : streamlit run app.py

Everything shown is taken from, or arithmetically derived from, that one CSV.
Derived values (growth %, shares, "rest of India" remainders) are labelled as such.
"""
import html as html_lib
import inspect
import re
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import seaborn as sns
import streamlit as st
import streamlit.components.v1 as components
from plotly.subplots import make_subplots
from scipy import stats

# ---------------------------------------------------------------------------------------------------------------------- #
# Page setup
# ---------------------------------------------------------------------------------------------------------------------- #
st.set_page_config(
    page_title="POCSO Cases in India - Dashboard",
    page_icon="\U0001f4ca",
    layout="wide",
    initial_sidebar_state="expanded",
)

DATA_PATH = Path(__file__).parent / "child_sexual_offence_cases_india_verified.csv"

ALL_OFF = "All POCSO offences"
SEC_OFF = "Sec 4 & 6 (penetrative)"
PALETTE = ["#4C78A8", "#F58518", "#54A24B", "#E45756", "#72B7B2",
           "#B279A2", "#9D755D", "#EECA3B", "#FF9DA6", "#BAB0AC"]
NAVY = "#1F2A44"

CSS = """
<style>
.block-container {padding-top: 1.6rem; padding-bottom: 2rem;}
@keyframes fadeUp {from {opacity: 0; transform: translateY(16px);} to {opacity: 1; transform: none;}}
@keyframes slideIn {from {opacity: 0; transform: translateX(-24px);} to {opacity: 1; transform: none;}}
div[data-testid="stPlotlyChart"], div[data-testid="stImage"], div[data-testid="stDataFrame"] {
    animation: fadeUp .8s ease both;
}
.hero {animation: slideIn .9s ease both; margin-bottom: .4rem;}
.hero h1 {
    font-size: 2.1rem; margin: 0; line-height: 1.2;
    background: linear-gradient(90deg, #4C78A8, #B279A2, #F58518);
    -webkit-background-clip: text; background-clip: text; color: transparent;
}
.hero p {opacity: .75; margin: .3rem 0 0 0;}
</style>
"""
st.markdown(CSS, unsafe_allow_html=True)


# ---------------------------------------------------------------------------------------------------------------------- #
# Small compatibility helpers (Streamlit's width API changed across versions)
# ---------------------------------------------------------------------------------------------------------------------- #
_PLOTLY_HAS_WIDTH = "width" in inspect.signature(st.plotly_chart).parameters


def show(fig, key):
    """Render a Plotly figure stretched to the column width."""
    if _PLOTLY_HAS_WIDTH:
```

```
        st.plotly_chart(fig, width="stretch", key=key)
    else:
        st.plotly_chart(fig, use_container_width=True, key=key)


# ---------------------------------------------------------------------------------------------------------------------- #
# Data layer
# ---------------------------------------------------------------------------------------------------------------------- #
def short_offence(text: str) -> str:
    t = str(text).lower()
    if "all offences" in t:
        return ALL_OFF
    if "sec 4 & 6" in t:
        return SEC_OFF
    if "share" in t:
        return "Victim gender share"
    return str(text)


@st.cache_data(show_spinner=False)
def load_data() -> pd.DataFrame:
    raw = pd.read_csv(DATA_PATH)
    raw["cases"] = pd.to_numeric(raw["total_cases_registered"], errors="coerce")
    raw["offence_short"] = raw["offence_category"].map(short_offence)
    raw["scope"] = np.where(raw["geography"].str.startswith("India"), "National", "State")
    raw["date"] = pd.to_datetime(raw["year"].astype(str) + "-12-31")
    return raw


def parse_gender_share(raw: pd.DataFrame):
    """The dataset holds one row with *percentages* of victims by gender (in its notes)."""
    row = raw[raw["cases"].isna()]
    if row.empty:
        return None
    r = row.iloc[0]
    nums = [float(x) for x in re.findall(r"(\d+(?:\.\d+)?)%", str(r["notes"]))]
    if len(nums) < 4:
        return None
    period = re.search(r"(\d{4})\s*-\s*(\d{4})", str(r["offence_category"]))
    return {
        "girls_range": (nums[0], nums[1]),
        "boys_range": (nums[2], nums[3]),
        "girls": float(np.mean(nums[:2])),
        "boys": float(np.mean(nums[2:4])),
        "period": f"{period.group(1)}-{period.group(2)}" if period else "",
        "source": str(r["source"]),
    }


def build_components(cases: pd.DataFrame) -> pd.DataFrame:
    """
    Decompose each reported national total into parts that add back up to it:
      * listed states (as reported)
      * 'Rest of India' = national total - listed states              (derived)
      * for 2020 (no state rows): Sec 4 & 6 + 'Other offences'        (derived)
    Years that have state rows but no national total are marked as partial.
    """
    allx = cases[cases["offence_short"] == ALL_OFF]
    nat = allx[allx["scope"] == "National"].set_index("year")["cases"]
    sts = allx[allx["scope"] == "State"]
    sec = cases[(cases["scope"] == "National") & (cases["offence_short"] == SEC_OFF)] \
        .set_index("year")["cases"]

    rows = []

    def add(year, comp, val, geo, derived):
        rows.append({"year": int(year), "component": comp, "cases": float(val),
                     "geography": geo, "derived": derived})

    for y in sorted(allx["year"].unique()):
        sy = sts[sts["year"] == y]
        for _, r in sy.iterrows():
            add(y, r["geography"], r["cases"], r["geography"], False)
        if y in nat.index:
            if len(sy):
                add(y, "Rest of India (derived)", nat[y] - sy["cases"].sum(), "India (all)", True)
            elif y in sec.index:
                add(y, "Sec 4 & 6 - penetrative (reported as 'more than')", sec[y], "India (all)", False
)
                add(y, "Other POCSO offences (derived)", nat[y] - sec[y], "India (all)", True)
            else:
                add(y, "India - all offences", nat[y], "India (all)", False)

    comp = pd.DataFrame(rows)
    nat_years = set(nat.index)
```

```
    comp["year_label"] = [str(y) if y in nat_years else f"{y} (listed states only)"
                          for y in comp["year"]]
    return comp


def change_table(f: pd.DataFrame) -> pd.DataFrame:
    """First-vs-latest reported year for every series with >= 2 reported years."""
    d = f[f["offence_short"] == ALL_OFF].sort_values("year")
    g = d.groupby("geography").agg(
        first_year=("year", "first"), last_year=("year", "last"),
        first_cases=("cases", "first"), last_cases=("cases", "last"), n=("year", "size"),
    )
    g = g[g["n"] >= 2].reset_index()
    if g.empty:
        return g
    span = (g["last_year"] - g["first_year"]).astype(float)
    g["change_pct"] = (g["last_cases"] / g["first_cases"] - 1) * 100
    g["cagr_pct"] = ((g["last_cases"] / g["first_cases"]) ** (1 / span) - 1) * 100
    return g


def compute_kpis(f: pd.DataFrame, total_records: int):
    cards = [{"label": "Records in view", "value": float(len(f)), "dec": 0,
              "sub": f"of {total_records} records with case counts"}]

    nat = f[(f["scope"] == "National") & (f["offence_short"] == ALL_OFF)].sort_values("year")
    if len(nat):
        last = nat.iloc[-1]
        cards.append({"label": "Latest national total", "value": float(last["cases"]), "dec": 0,
                      "sub": f"{int(last['year'])} · all POCSO offences"})
    else:
        cards.append({"label": "Latest national total", "value": None, "sub": "No national record in vie
    w"})

    if len(nat) >= 2:
        a, b = nat.iloc[0], nat.iloc[-1]
        yrs = int(b["year"] - a["year"])
        pct = (b["cases"] / a["cases"] - 1) * 100
        cagr = ((b["cases"] / a["cases"]) ** (1 / yrs) - 1) * 100
        cards.append({"label": "Growth in registered cases", "value": float(pct), "dec": 1,
                      "prefix": "+" if pct >= 0 else "", "suffix": "%",
                      "sub": f"{int(a['year'])} → {int(b['year'])} · CAGR {cagr:.1f}%/yr"})
    else:
        cards.append({"label": "Growth in registered cases", "value": None,
                      "sub": "Needs ≥2 national years in view"})

    sts = f[f["scope"] == "State"]
    if len(sts):
        top = sts.loc[sts["cases"].idxmax()]
        cards.append({"label": "Highest state record", "value": float(top["cases"]), "dec": 0,
                      "sub": f"{top['geography']} · {int(top['year'])}"})
    else:
        cards.append({"label": "Highest state record", "value": None, "sub": "No state record in view"})

    best = None
    nat_s = nat.set_index("year")["cases"]
    for y, tot in nat_s.items():
        sy = f[(f["scope"] == "State") & (f["year"] == y) & (f["offence_short"] == ALL_OFF)]
        if len(sy) and (best is None or len(sy) > best[1]):
            best = (int(y), len(sy), sy["cases"].sum() / tot * 100)
    if best:
        cards.append({"label": "Listed states' share of national", "value": float(best[2]), "dec": 1,
                      "suffix": "%", "sub": f"{best[1]} state(s) listed · {best[0]}"})
    else:
        cards.append({"label": "Listed states' share of national", "value": None,
                      "sub": "Needs national + state rows"})
    return cards


KPI_TEMPLATE = """
<style>
  body {margin:0; font-family: "Source Sans Pro", system-ui, sans-serif; background: transparent;}
  .grid {display:grid; grid-template-columns: repeat(auto-fit, minmax(175px, 1fr)); gap: 12px;}
  .card {background: linear-gradient(135deg, #1F2A44 0%, #2E3F6B 100%); color: #fff;
         border-radius: 14px; padding: 14px 16px; border-left: 5px solid #F58518;
         box-shadow: 0 4px 14px rgba(0,0,0,.18); opacity: 0; transform: translateY(14px);
         animation: pop .7s ease forwards;}
  .lab {font-size: 12px; letter-spacing: .04em; text-transform: uppercase; opacity: .75;}
  .val {font-size: 30px; font-weight: 700; margin: 4px 0 2px 0; font-variant-numeric: tabular-nums;}
  .sub {font-size: 12px; opacity: .8;}
  @keyframes pop {to {opacity: 1; transform: none;}}
</style>
<div class="grid">__ITEMS__</div>
<script>
  document.querySelectorAll('.val[data-target]').forEach(function (el) {
```

```
    var target = parseFloat(el.dataset.target), dec = parseInt(el.dataset.dec || '0'),
         pre = el.dataset.prefix || '', suf = el.dataset.suffix || '', dur = 1500, t0 = performance.now()
    ;
    function fmt(v) {
        return pre + v.toLocaleString('en-IN', {minimumFractionDigits: dec, maximumFractionDigits: dec}) +
     suf;
    }
    function tick(now) {
      var p = Math.min((now - t0) / dur, 1), e = 1 - Math.pow(1 - p, 3);
      el.textContent = fmt(target * e);
      if (p < 1) requestAnimationFrame(tick);
    }
    requestAnimationFrame(tick);
  });
</script>
"""


def render_kpis(cards):
    items = []
    for i, c in enumerate(cards):
        label, sub = html_lib.escape(c["label"]), html_lib.escape(c["sub"])
        if c["value"] is None:
            val = '<div class="val">-</div>'
        else:
            val = (f'<div class="val" data-target="{c["value"]}" data-dec="{c.get("dec", 0)}" '
                   f'data-prefix="{c.get("prefix", "")}" data-suffix="{c.get("suffix", "")}">0</div>')
        items.append(f'<div class="card" style="animation-delay:{i * 0.1:.1f}s">'
                     f'<div class="lab">{label}</div>{val}<div class="sub">{sub}</div></div>')
    page = KPI_TEMPLATE.replace("__ITEMS__", "".join(items))
    try:
        components.html(page, height=135)
    except Exception:  # very defensive fallback: static cards without count-up
        cols = st.columns(len(cards))
        for col, c in zip(cols, cards):
            col.metric(c["label"], "-" if c["value"] is None else f"{c['value']:,.{c.get('dec', 0)}f}",
    c["sub"])


# ---------------------------------------------------------------------------------------------------------------------- #
# Chart helpers
# ---------------------------------------------------------------------------------------------------------------------- #
def base_layout(fig, title=None, height=430):
    fig.update_layout(
        title=dict(text=title, x=0, font=dict(size=16)) if title else None,
        height=height,
        margin=dict(l=10, r=10, t=64 if title else 24, b=10),
        legend=dict(orientation="h", y=1.1, x=0),
    )
    return fig


def update_all_traces(fig, **kw):
    """update_traces() skips animation frames, so patch those too."""
    fig.update_traces(**kw)
    for fr in fig.frames:
        for tr in fr.data:
            tr.update(**kw)


def play_controls(fig, frame_names, duration, active=0, prefix="Year: "):
    """Own play/pause buttons + slider so the sidebar speed control applies to every animation."""
    if not frame_names:
        return fig
    anim = {"frame": {"duration": duration, "redraw": True}, "mode": "immediate",
            "fromcurrent": False,
            "transition": {"duration": int(duration * 0.5), "easing": "cubic-in-out"}}
    fig.update_layout(
        updatemenus=[dict(
            type="buttons", direction="left", showactive=False,
            pad={"r": 10, "t": 70}, x=0.1, xanchor="right", y=0, yanchor="top",
            buttons=[
                dict(label="\u25b6 Play", method="animate", args=[None, anim]),
                dict(label="\u23f8 Pause", method="animate",
                     args=[[None], {"frame": {"duration": 0, "redraw": False},
                                    "mode": "immediate", "transition": {"duration": 0}}]),
            ])],
        sliders=[dict(
            active=active, x=0.1, len=0.9, xanchor="left", y=0, yanchor="top",
            pad={"b": 10, "t": 60}, currentvalue={"prefix": prefix},
            steps=[dict(label=n, method="animate",
                        args=[[n], {"frame": {"duration": duration, "redraw": True},
                                    "mode": "immediate",
                                    "transition": {"duration": int(duration * 0.5)}}])
                   for n in frame_names])],
```

```
    )
    return fig


def axis_range(values_min, values_max, log):
    if log:
        return dict(type="log", range=[np.log10(max(values_min * 0.55, 1)), np.log10(values_max * 2.4)])
    return dict(type="linear", range=[0, values_max * 1.18])


# ---------------------------------------------------------------------------------------------------------------- #
# Charts
# ---------------------------------------------------------------------------------------------------------------- #
def fig_bar_race(f, mode, speed, off_colors):
    d = f.copy()
    if mode == "States only":
        d = d[d["scope"] == "State"]
    elif mode == "National only":
        d = d[d["scope"] == "National"]
    if d.empty:
        return None
    log = mode.startswith("All")
    years = sorted(int(y) for y in d["year"].unique())
    order = [g for g in GEO_ORDER if g in set(d["geography"])]
    fig = px.bar(
        d, x="geography", y="cases", color="offence_short", animation_frame="year",
        barmode="group", text_auto=",.0f",
        category_orders={"year": years, "geography": order},
        color_discrete_map=off_colors,
        labels={"cases": "Registered cases", "geography": "", "offence_short": "Offence category"},
    )
    update_all_traces(fig, textposition="outside", cliponaxis=False)
    fig.update_yaxes(**axis_range(d["cases"].min(), d["cases"].max(), log))
    base_layout(fig, "Registered cases by geography - press \u25b6 Play to step through the years", 520)
    return play_controls(fig, [fr.name for fr in fig.frames], speed)


def fig_donut(share):
    labels = ["Girls", "Boys", "Unspecified / rounding"]
    values = [share["girls"], share["boys"], max(0.0, 100 - share["girls"] - share["boys"])]
    custom = [f"{share['girls_range'][0]}%-{share['girls_range'][1]}%",
              f"{share['boys_range'][0]}%-{share['boys_range'][1]}%", "remainder"]
    fig = go.Figure(go.Pie(
        labels=labels, values=values, hole=0.62, sort=False, customdata=custom,
        marker=dict(colors=["#E45756", "#4C78A8", "#BAB0AC"], line=dict(color="white", width=2)),
        textinfo="label+percent", textposition="outside",
        hovertemplate="<b>%{label}</b><br>Reported range: %{customdata}<extra></extra>",
        pull=[0.03, 0, 0],
    ))
    fig.add_annotation(text=f"Victims<br><b>{share['period']}</b>", showarrow=False, font=dict(size=15))
    base_layout(fig, "Victim gender split (donut)", 430)
    fig.update_layout(showlegend=False)
    return fig


def fig_pie(comp_f, year_label):
    d = comp_f[comp_f["year_label"] == year_label]
    colors = [PALETTE[i % len(PALETTE)] for i in range(len(d))]
    fig = go.Figure(go.Pie(
        labels=d["component"], values=d["cases"], sort=False,
        marker=dict(colors=colors, line=dict(color="white", width=2)),
        textinfo="percent", hovertemplate="<b>%{label}</b><br>%{value:,.0f} cases<br>%{percent}<extra></
extra>",
    ))
    base_layout(fig, f"Composition of registered cases - {year_label} (pie)", 430)
    fig.update_layout(legend=dict(orientation="v", x=1.0, y=0.5))
    return fig


def fig_sunburst(comp_f):
    fig = px.sunburst(
        comp_f, path=["year_label", "component"], values="cases", color="year_label",
        color_discrete_sequence=PALETTE,
    )
    fig.update_traces(
        insidetextorientation="radial", textinfo="label+percent parent",
        hovertemplate="<b>%{label}</b><br>%{value:,.0f} cases<extra></extra>",
    )
    return base_layout(fig, "Year → component breakdown (sunburst)", 520)


def fig_line_draw(f, log, speed, geo_colors):
    d = f.sort_values("year").copy()
    multi = d["offence_short"].nunique() > 1
    d["series"] = np.where(multi, d["geography"] + " · " + d["offence_short"], d["geography"])
```

```
    series = list(dict.fromkeys(d["series"]))
    years = sorted(int(y) for y in d["year"].unique())
    dashes = {ALL_OFF: "solid", SEC_OFF: "dot"}

    def traces_upto(y):
        out = []
        for s in series:
            sd = d[(d["series"] == s) & (d["year"] <= y)]
            geo, off = (sd["geography"].iloc[0], sd["offence_short"].iloc[0]) if len(sd) else \
                (d[d["series"] == s]["geography"].iloc[0], d[d["series"] == s]["offence_short"].iloc[0])
            out.append(go.Scatter(
                x=sd["year"], y=sd["cases"], mode="lines+markers", name=s,
                line=dict(color=geo_colors.get(geo, NAVY), width=3.5, dash=dashes.get(off, "solid")),
                marker=dict(size=10),
                hovertemplate="%{x}: %{y:,.0f} cases<extra>" + s + "</extra>"))
        return out

    frames = [go.Frame(data=traces_upto(y), name=str(y)) for y in years]
    fig = go.Figure(data=traces_upto(years[-1]), frames=frames)
    fig.update_xaxes(tickmode="array", tickvals=years, range=[years[0] - 0.4, years[-1] + 0.4],
                     title="Year")
    fig.update_yaxes(title="Registered cases", **axis_range(d["cases"].min(), d["cases"].max(), log))
    base_layout(fig, "Registered cases over time (line) - \u25b6 Play redraws the lines year by year", 5
    20)
    return play_controls(fig, [str(y) for y in years], speed, active=len(years) - 1)


def series_options(f):
    opts = {}
    for (geo, off), g in f.groupby(["geography", "offence_short"]):
        if len(g) >= 2:
            opts[geo if off == ALL_OFF else f"{geo} · {off}"] = g.sort_values("year")
    return opts


def fig_timeseries(g, label):
    x, y = g["year"].astype(float).to_numpy(), g["cases"].to_numpy(dtype=float)
    yoy = g["cases"].pct_change() * 100
    fig = make_subplots(specs=[[{"secondary_y": True}]])
    fig.add_bar(
        x=g["date"], y=yoy, name="Change vs previous reported year (%)", secondary_y=True,
        marker_color="rgba(245,133,24,0.45)", width=1000 * 3600 * 24 * 70,
        text=[f"{v:+.1f}%" if pd.notna(v) else "" for v in yoy], textposition="outside",
        hovertemplate="%{y:+.1f}%<extra>Change</extra>")
    fig.add_scatter(
        x=g["date"], y=g["cases"], mode="lines+markers", name="Registered cases",
        line=dict(color="#4C78A8", width=4), marker=dict(size=11), fill="tozeroy",
        fillcolor="rgba(76,120,168,0.15)", hovertemplate="%{x|%Y}: %{y:,.0f}<extra></extra>")
    info = {"n": len(g), "slope": None, "r2": None}
    if len(g) >= 3:
        res = stats.linregress(x, y)
        fig.add_scatter(x=g["date"], y=res.intercept + res.slope * x, mode="lines", name="Linear trend (
    scipy)",
                        line=dict(color="#E45756", width=2.5, dash="dash"))
        info.update(slope=float(res.slope), r2=float(res.rvalue ** 2))
    # label each tick with its data year (data points sit on 31 Dec, so automatic yearly ticks would rea
    d a year late)
    fig.update_xaxes(rangeslider_visible=True, tickmode="array", tickvals=list(g["date"]),
                     ticktext=[str(int(y)) for y in g["year"]])
    fig.update_yaxes(title_text="Registered cases", secondary_y=False, rangemode="tozero")
    fig.update_yaxes(title_text="Change (%)", secondary_y=True, showgrid=False)
    return base_layout(fig, f"Time series - {label}", 520), info


def fig_hist(values, nbins):
    counts, edges = np.histogram(values, bins=nbins)
    centers, width = (edges[:-1] + edges[1:]) / 2, edges[1] - edges[0]
    fig = go.Figure(go.Bar(
        x=centers, y=counts, width=width * 0.92, name="Records", marker_color="#4C78A8",
        customdata=np.c_[edges[:-1], edges[1:]],
        hovertemplate="%{customdata[0]:,.0f} - %{customdata[1]:,.0f} cases<br>Records: %{y}<extra></extr
    a>"))
    if len(values) >= 3 and np.ptp(values) > 0:
        kde = stats.gaussian_kde(values)
        xs = np.linspace(edges[0] - width * 0.5, edges[-1] + width * 0.5, 200)
        fig.add_scatter(x=xs, y=kde(xs) * len(values) * width, mode="lines", name="Density (KDE, scaled)
    ",
                        line=dict(color="#F58518", width=3))
    fig.update_layout(bargap=0.04, xaxis_title="Registered cases per record", yaxis_title="Number of rec
    ords")
    return base_layout(fig, "Distribution of case counts (histogram)", 430)


def fig_rank_scatter(f, speed, geo_colors):
    d = f[f["scope"] == "State"].copy()
```

```
    if d.empty:
        return None
    d["rank"] = d.groupby("year")["cases"].rank(ascending=False, method="first").astype(int)
    years = sorted(int(y) for y in d["year"].unique())
    fig = px.scatter(
        d, x="rank", y="cases", size="cases", color="geography", text="geography",
        animation_frame="year", size_max=46, log_y=True,
        category_orders={"year": years}, color_discrete_map=geo_colors,
        labels={"rank": "Rank within year (1 = most cases)", "cases": "Registered cases"})
    update_all_traces(fig, textposition="top center")
    fig.update_xaxes(range=[0.4, d["rank"].max() + 0.8], dtick=1)
    fig.update_yaxes(range=[np.log10(d["cases"].min() * 0.6), np.log10(d["cases"].max() * 1.9)])
    base_layout(fig, "Rank vs cases for states (animated scatter, log scale)", 520)
    fig.update_layout(showlegend=False)
    return play_controls(fig, [fr.name for fr in fig.frames], speed)


def fig_paired(ch):
    lo = min(ch["first_cases"].min(), ch["last_cases"].min()) * 0.7
    hi = max(ch["first_cases"].max(), ch["last_cases"].max()) * 1.5
    fig = go.Figure()
    fig.add_scatter(x=[lo, hi], y=[lo, hi], mode="lines", name="No change (y = x)",
                    line=dict(color="grey", dash="dot"))
    for i, r in ch.reset_index(drop=True).iterrows():
        fig.add_scatter(
            x=[r["first_cases"]], y=[r["last_cases"]], mode="markers+text", text=[r["geography"]],
            textposition="top center", name=r["geography"],
            marker=dict(size=18, color=PALETTE[i % len(PALETTE)], line=dict(color="white", width=1.5)),
            hovertemplate=(f"<b>{r['geography']}</b><br>{int(r['first_year'])}: {r['first_cases']:,.0f}"
                           f"<br>{int(r['last_year'])}: {r['last_cases']:,.0f}"
                           f"<br>Change: {r['change_pct']:+.1f}%<extra></extra>"))
    fig.update_xaxes(type="log", range=[np.log10(lo), np.log10(hi)], title="Cases in first reported year
    ")
    fig.update_yaxes(type="log", range=[np.log10(lo), np.log10(hi)], title="Cases in latest reported yea
    r")
    base_layout(fig, "First vs latest reported year (scatter) - above the dotted line = increase", 430)
    fig.update_layout(showlegend=False)
    return fig


def fig_radar(f):
    d = f[(f["scope"] == "State") & (f["offence_short"] == ALL_OFF)]
    states = sorted(d["geography"].unique(), key=lambda g: -d[d["geography"] == g]["cases"].max())
    if len(states) < 3:
        return None
    fig = go.Figure()
    for i, y in enumerate(sorted(d["year"].unique())):
        dy = d[d["year"] == y].set_index("geography")["cases"]
        r = [float(dy[s]) if s in dy.index else None for s in states]
        complete = all(v is not None for v in r)
        fig.add_trace(go.Scatterpolar(
            r=r + [r[0]], theta=states + [states[0]], name=str(int(y)), mode="lines+markers",
            fill="toself" if complete else "none", opacity=0.85, connectgaps=False,
            line=dict(color=PALETTE[i % len(PALETTE)], width=3), marker=dict(size=8),
            hovertemplate="%{theta}: %{r:,.0f}<extra>" + str(int(y)) + "</extra>"))
    fig.update_layout(polar=dict(radialaxis=dict(visible=True, range=[0, d["cases"].max() * 1.15])))
    return base_layout(fig, "State profile by year (radar)", 480)


def fig_heatmap(f):
    d = f[(f["scope"] == "State") & (f["offence_short"] == ALL_OFF)]
    if d.empty:
        return None
    pv = d.pivot_table(index="geography", columns="year", values="cases", aggfunc="sum")
    pv = pv.loc[pv.max(axis=1).sort_values(ascending=False).index]
    fig, ax = plt.subplots(figsize=(7, max(2.6, 0.55 * len(pv) + 1.2)))
    ax.set_facecolor("#EEF0F4")
    sns.heatmap(pv, annot=True, fmt=",.0f", cmap="YlOrRd", linewidths=0.8, linecolor="white",
                cbar_kws={"label": "Registered cases"}, mask=pv.isna(), ax=ax)
    ax.set_xlabel("")
    ax.set_ylabel("")
    ax.set_title("State × year heatmap (seaborn) - grey = not in dataset", loc="left", fontsize=11)
    fig.tight_layout()
    return fig


# ---------------------------------------------------------------------------------------------------------------------- #
# Load data, build global lookups
# ---------------------------------------------------------------------------------------------------------------------- #
if not DATA_PATH.exists():
    st.error(f"Dataset not found at `{DATA_PATH}`. Keep the CSV in the same folder as app.py.")
    st.stop()

raw = load_data()
cases = raw[raw["cases"].notna()].copy()
```

```
share_info = parse_gender_share(raw)
comp_all = build_components(cases)

NUMERIC_COLS = ["victims_girls", "victims_boys", "victims_other_gender",
                "cases_solved_chargesheeted", "cases_convicted"]
empty_cols = [c for c in NUMERIC_COLS if c in raw.columns and raw[c].isna().all()]

GEO_ORDER = list(cases.sort_values("cases", ascending=False)["geography"].drop_duplicates())
ALL_GEOS = list(cases["geography"].drop_duplicates())
GEO_COLORS = {g: (NAVY if g.startswith("India") else PALETTE[i % len(PALETTE)])
              for i, g in enumerate(ALL_GEOS)}
OFF_COLORS = {ALL_OFF: "#4C78A8", SEC_OFF: "#E45756"}

years_all = [int(y) for y in sorted(cases["year"].unique())]
offs_all = list(cases["offence_short"].drop_duplicates())
lo_cases, hi_cases = int(cases["cases"].min()), int(cases["cases"].max())

# ---------------------------------------------------------------------------------------------------------------- #
# Sidebar - filters (no file upload by design)
# ---------------------------------------------------------------------------------------------------------------- #
DEFAULTS = {"f_years": years_all, "f_geos": ALL_GEOS, "f_offs": offs_all, "f_range": (lo_cases, hi_cases
    )}


def reset_filters():
    for k, v in DEFAULTS.items():
        st.session_state[k] = list(v) if isinstance(v, list) else v


with st.sidebar:
    st.header("\U0001f39b\ufe0f Filters")
    st.multiselect("Year", years_all, default=years_all, key="f_years")
    st.multiselect("Geography", ALL_GEOS, default=ALL_GEOS, key="f_geos")
    st.multiselect("Offence category", offs_all, default=offs_all, key="f_offs")
    st.slider("Registered cases (range)", lo_cases, hi_cases, value=(lo_cases, hi_cases), step=1, key="f
    _range")
    st.button("\u21ba Reset filters", on_click=reset_filters)
    st.divider()
    st.header("\U0001f3ac Animation")
    speed = st.slider("Frame duration (ms)", 300, 2500, 900, step=100,
                      help="Lower = faster. Applies to every animated chart.")
    st.caption("Use \u25b6 Play under the animated charts, or drag their year slider.")

mask = (cases["year"].isin(st.session_state["f_years"])
        & cases["geography"].isin(st.session_state["f_geos"])
        & cases["offence_short"].isin(st.session_state["f_offs"])
        & cases["cases"].between(*st.session_state["f_range"]))
f = cases[mask].sort_values(["year", "geography"]).copy()

comp_f = comp_all[
    comp_all["year"].isin(st.session_state["f_years"])
    & comp_all["geography"].isin(st.session_state["f_geos"])
    & comp_all["cases"].between(*st.session_state["f_range"])
].copy() if ALL_OFF in st.session_state["f_offs"] else comp_all.iloc[0:0].copy()

st.sidebar.divider()
st.sidebar.caption(f"Showing **{len(f)}** of {len(cases)} records")

# ---------------------------------------------------------------------------------------------------------------- #
# Header + KPIs
# ---------------------------------------------------------------------------------------------------------------- #
st.markdown(
    f"""<div class="hero"><h1>Child Sexual Offence Cases in India</h1>
    <p>POCSO Act registered-case figures, {years_all[0]}-{years_all[-1]} · NCRB data as compiled in the
    bundled CSV
    ({len(cases)} case-count records)</p></div>""",
    unsafe_allow_html=True,
)

if f.empty:
    st.warning("No records match the current filters. Widen the filters or press **Reset filters**.")
    st.stop()

render_kpis(compute_kpis(f, len(cases)))

tab_over, tab_trend, tab_geo, tab_dist, tab_data = st.tabs(
    ["\U0001f4c8 Overview", "\u23f1 Trends", "\U0001f5fa Geography", "\U0001f4ca Distribution & relation
    ships", "\U0001f5c2 Data & notes"])

# ---------------------------------------------------- Overview ---------------------------------------------------- #
with tab_over:
    c1, c2 = st.columns([2, 1])
    with c1:
        mode = st.radio("Scope", ["All records (log scale)", "States only", "National only"],
                        horizontal=True, key="bar_mode")
```

```
650          fig = fig_bar_race(f, mode, speed, OFF_COLORS)
651          if fig is None:
652              st.info("No records for this scope under the current filters.")
653          else:
654              show(fig, "bar_race")
655              if mode.startswith("All"):
656                  st.caption("Log scale: national totals are ~10x larger than single-state counts, so bar
     "
657                             "length is not proportional to value here.")
658      with c2:
659          if share_info:
660              show(fig_donut(share_info), "donut")
661              st.caption(f"Midpoint of reported ranges (girls {share_info['girls_range'][0]}-"
662                         f"{share_info['girls_range'][1]}%, boys {share_info['boys_range'][0]}-"
663                         f"{share_info['boys_range'][1]}%). Percentages, not counts; not affected by filte
     rs.")
664          else:
665              st.info("No gender-share information in the dataset.")
666
667      st.subheader("Composition")
668      if comp_f.empty:
669          st.info("Composition charts need 'All POCSO offences' selected and at least one matching record.
     ")
670      else:
671          p1, p2 = st.columns(2)
672          with p1:
673              labels = list(dict.fromkeys(comp_f.sort_values("year")["year_label"]))
674              counts = comp_f.groupby("year_label")["component"].count()
675              default_idx = labels.index(counts.idxmax()) if len(labels) else 0
676              ylab = st.selectbox("Year for pie chart", labels, index=default_idx, key="pie_year")
677              show(fig_pie(comp_f, ylab), "pie")
678          with p2:
679              show(fig_sunburst(comp_f), "sunburst")
680          st.caption("'Rest of India' and 'Other POCSO offences' are **derived** (national total minus the
      parts listed "
681                     "in the dataset). Years labelled 'listed states only' have no national total in the d
     ataset.")
682
683  # ---------------------------------------------------- Trends ---------------------------------------------------- #
684  with tab_trend:
685      log_line = st.toggle("Log scale", value=False, key="line_log")
686      show(fig_line_draw(f, log_line, speed, GEO_COLORS), "line_draw")
687
688      st.subheader("Time series with trend")
689      opts = series_options(f)
690      if not opts:
691          st.info("A time series needs a series with at least 2 reported years. Widen the year filter.")
692      else:
693          label = st.selectbox("Series", list(opts.keys()), key="ts_series")
694          fig_ts, info = fig_timeseries(opts[label], label)
695          show(fig_ts, "timeseries")
696          if info["slope"] is not None:
697              st.caption(f"Linear fit (scipy.stats.linregress): **{info['slope']:+,.0f} cases per year**,
     "
698                         f"R² = {info['r2']:.2f}, n = {info['n']} years. Indicative only - not a forecast.
     ")
699          else:
700              st.caption(f"Only {info['n']} reported years, so no trend line is fitted.")
701
702      ch = change_table(f)
703      if not ch.empty:
704          st.subheader("Change between first and latest reported year")
705          tbl = ch.rename(columns={"geography": "Geography", "first_year": "From", "last_year": "To",
706                                   "first_cases": "Cases (from)", "last_cases": "Cases (to)",
707                                   "change_pct": "Change %", "cagr_pct": "CAGR %/yr"}).drop(columns="n")
708          st.dataframe(tbl.style.format({"Cases (from)": "{:,.0f}", "Cases (to)": "{:,.0f}",
709                                         "Change %": "{:+.1f}", "CAGR %/yr": "{:+.1f}"}), hide_index=True)
710
711  # ---------------------------------------------------- Geography ---------------------------------------------------- #
712  with tab_geo:
713      g1, g2 = st.columns(2)
714      with g1:
715          fr = fig_radar(f)
716          if fr is None:
717              st.info("A radar needs at least 3 states in view (all-offence records).")
718          else:
719              show(fr, "radar")
720              st.caption("Each axis is a state; each polygon is a year. Years with fewer than all states "
721                         "show as points (the dataset has no value for the missing states).")
722      with g2:
723          hm = fig_heatmap(f)
724          if hm is None:
725              st.info("No state-level records in view.")
726          else:
727              st.pyplot(hm)
```

```
            plt.close(hm)

# ------------------------------------ Distribution & relationships ---------------------------------------- #
with tab_dist:
    d1, d2 = st.columns([1, 2])
    with d1:
        scope_h = st.selectbox("Records to include", ["State-level records", "National records", "All records"],
                               key="hist_scope")
    with d2:
        nb = st.slider("Histogram bins", 3, 12, 5, key="hist_bins")
    hv = f if scope_h == "All records" else f[f["scope"] == ("State" if scope_h.startswith("State") else "National")]
    vals = hv["cases"].to_numpy(dtype=float)
    if len(vals) < 2:
        st.info("A histogram needs at least 2 records. Change the scope or widen the filters.")
    else:
        show(fig_hist(vals, nb), "hist")
        m1, m2, m3, m4 = st.columns(4)
        m1.metric("Records (n)", f"{len(vals)}")
        m2.metric("Mean", f"{vals.mean():,.0f}")
        m3.metric("Median", f"{np.median(vals):,.0f}")
        m4.metric("Skewness", f"{stats.skew(vals):.2f}" if len(vals) >= 3 else "n/a")
        st.caption(f"Very small sample (n = {len(vals)}): the shape is descriptive, not a statistical finding.")

    s1, s2 = st.columns([3, 2])
    with s1:
        fs = fig_rank_scatter(f, speed, GEO_COLORS)
        if fs is None:
            st.info("No state-level records in view for the animated scatter.")
        else:
            show(fs, "rank_scatter")
    with s2:
        ch2 = change_table(f)
        if ch2.empty:
            st.info("The first-vs-latest scatter needs a series with ≥2 reported years.")
        else:
            show(fig_paired(ch2), "paired")

# ------------------------------------------------------ Data & notes ------------------------------------------------------ #
with tab_data:
    st.subheader("Records in view")
    cols = ["year", "geography", "offence_category", "cases", "source", "notes"]
    view = f[cols].rename(columns={"cases": "total_cases_registered"})
    st.dataframe(view, hide_index=True)
    st.download_button("\u2b07 Download filtered records (CSV)", view.to_csv(index=False).encode("utf-8"),
                       file_name="pocso_filtered_records.csv", mime="text/csv")

    st.subheader("About this data")
    notes = [
        f"The dataset has **{len(raw)} rows**; {len(cases)} carry a case count, "
        f"{len(raw) - len(cases)} carries only victim-gender percentages (donut chart).",
        "Counts are **registered cases** (not victims, not convictions). A rise can reflect more reporting "
        "and registration as well as more offences.",
        "Coverage is patchy: all-India totals exist for 2020-2022; state figures only for 2021 (5 states), "
        "Uttar Pradesh 2022 and Karnataka 2023.",
        "The 2020 'Sec 4 & 6' value is reported only as 'more than 28,000', so it is a lower bound.",
    ]
    if empty_cols:
        notes.append("Columns present but **entirely empty** (so not charted): "
                     + ", ".join(f"`{c}`" for c in empty_cols) + ".")
    st.markdown("\n".join(f"- {n}" for n in notes))
    st.markdown("**Sources named in the dataset**")
    st.markdown("\n".join(f"- {s}" for s in sorted(raw["source"].dropna().unique())))

st.divider()
st.caption("Built only from the bundled CSV. If a child is in danger in India, CHILDLINE 1098 is the 24×7 helpline.")
```

B SAMPLE DATASET

All 12 records of the dataset, exactly as bundled with the application. The columns for victim counts by gender, charge-sheeted cases and convictions are empty in every row and are omitted here.

Table B.1 : Dataset records

Year	Geography	Offence category	Cases	Source	Notes
2020	India (all)	POCSO Act - all offences	47,221	NCRB Crime in India 2021 (via Deccan Herald 30-Aug-2022)	
2020	India (all)	POCSO Act Sec 4 & 6 (penetrative sexual assault / rape)	28,000	NCRB Crime in India 2020 (via Factly)	Reported only as 'more than 28000'; ~99% of victims were girls
2021	India (all)	POCSO Act - all offences	53,874	NCRB Crime in India 2021 (via Deccan Herald 30-Aug-2022)	
2022	India (all)	POCSO Act - all offences	63,414	NCRB Crime in India 2022 (via NewsClick)	
2021	Uttar Pradesh	POCSO Act - all offences	7,129	NCRB Crime in India 2021 (via Deccan Herald)	
2021	Maharashtra	POCSO Act - all offences	6,200	NCRB Crime in India 2021 (via Deccan Herald)	
2021	Madhya Pradesh	POCSO Act - all offences	6,070	NCRB Crime in India 2021 (via Deccan Herald)	
2021	Tamil Nadu	POCSO Act - all offences	4,465	NCRB Crime in India 2021 (via Deccan Herald)	
2021	Karnataka	POCSO Act - all offences	2,813	NCRB Crime in India 2021 (via Deccan Herald)	
2022	Uttar Pradesh	POCSO Act - all offences	8,136	NCRB Crime in India 2022 (via NewsClick)	
2023	Karnataka	POCSO Act - all offences	3,878	NCRB Crime in India 2023 (via Deccan Herald)	
2020	India (all)	Girls' share of POCSO victims 2017-2020	–	NCRB (via Factly)	Girls 97.1%-97.6% of POCSO victims; boys 2.3%-2.8% (percentages not counts)

C ADDITIONAL SCREENSHOTS

The screenshots below were taken from the running application (`streamlit run app.py`, served at localhost:8501, dark theme) with all filters at their default values. Browser tabs and address bar have been cropped out. Each figure follows the order of the dashboard: sidebar, header and key indicators, then the five tabs.

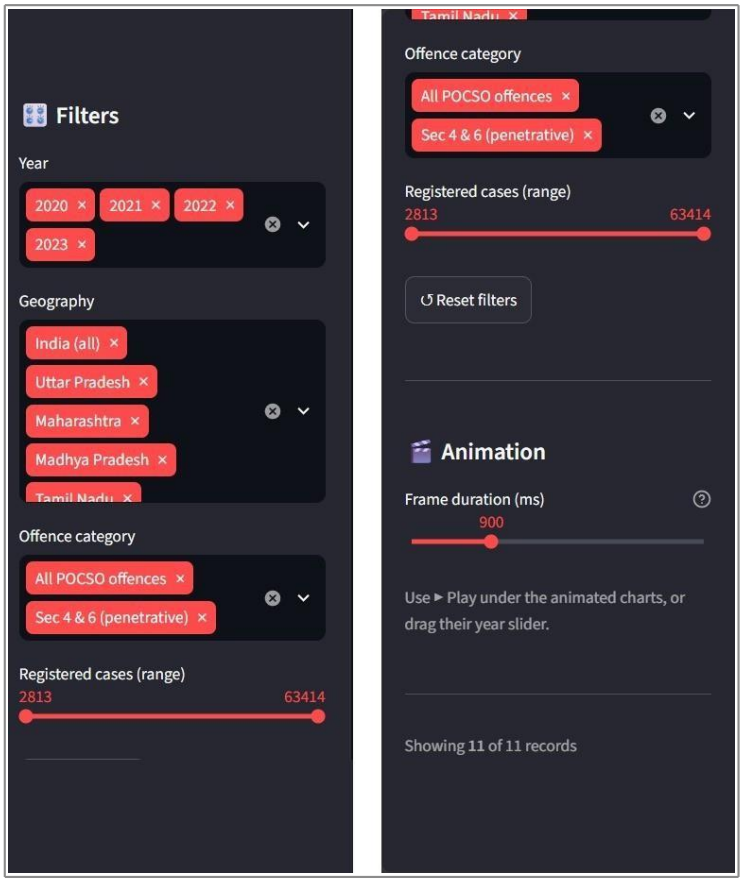


Figure C.1 : Sidebar filters and animation controls

Left: the Year, Geography and Offence category multi-selects, the registered-cases range slider and the Reset filters button. Right: the Frame duration slider, which sets the speed of every animated chart, and the live count of records in view (11 of 11). There is no file-upload control.

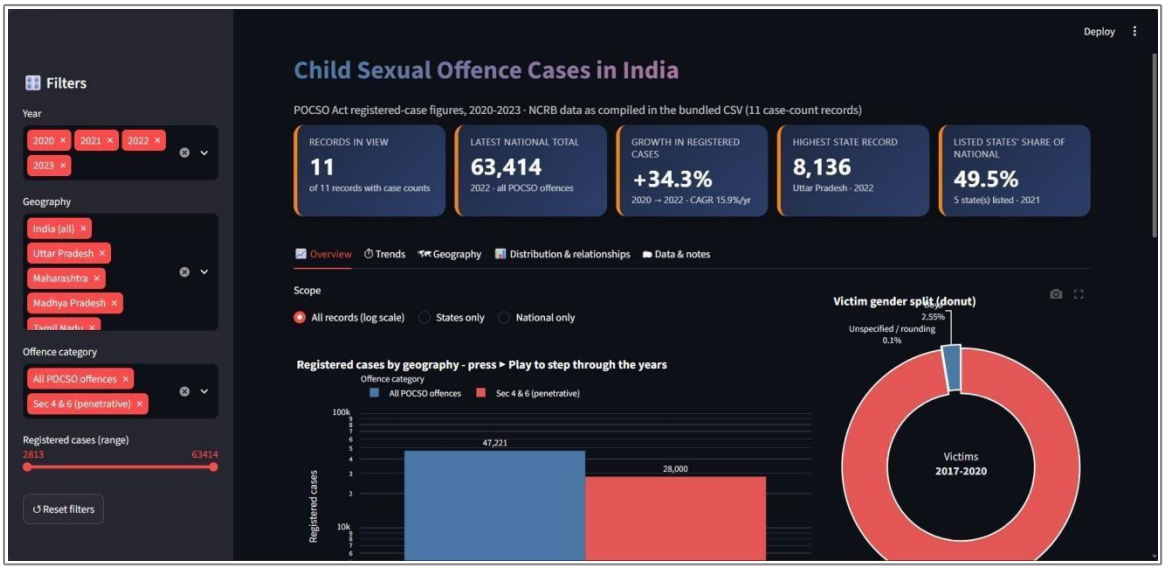


Figure C.2 : Dashboard header and key-indicator cards

The page title, the five animated key-indicator cards (11 records in view; latest national total 63,414 in 2022; growth +34.3% from 2020 to 2022 with a CAGR of 15.9% per year; highest state record 8,136 for Uttar Pradesh in 2022; listed states' share of the national total 49.5%), the five tabs and the start of the Overview tab.



Figure C.3 : Overview tab: animated bar chart and donut chart

The scope selector, the grouped bar chart on a log scale with its Play / Pause buttons and year slider (frame 2020, showing the all-offence total 47,221 and the Sec 4 & 6 value 28,000), and the donut chart of the victim gender split with its caption.

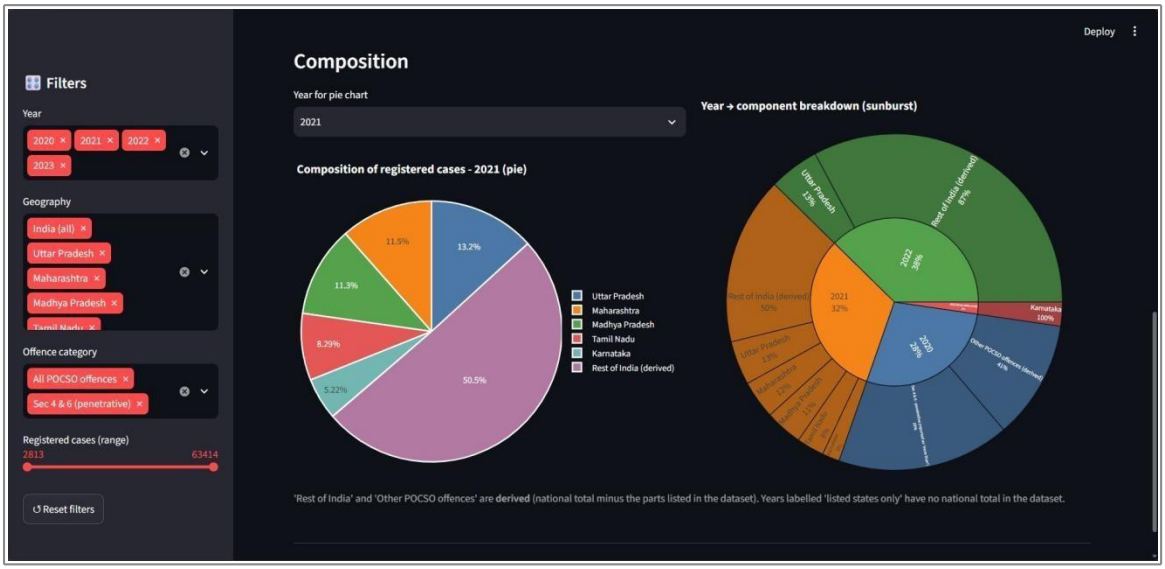


Figure C.4 : Overview tab: composition (pie chart and sunburst)

The pie chart for the selected year (2021) beside the year-to-component sunburst. The derived "Rest of India" slice is 50.5% of the 2021 total, and a note below the charts explains which components are derived.

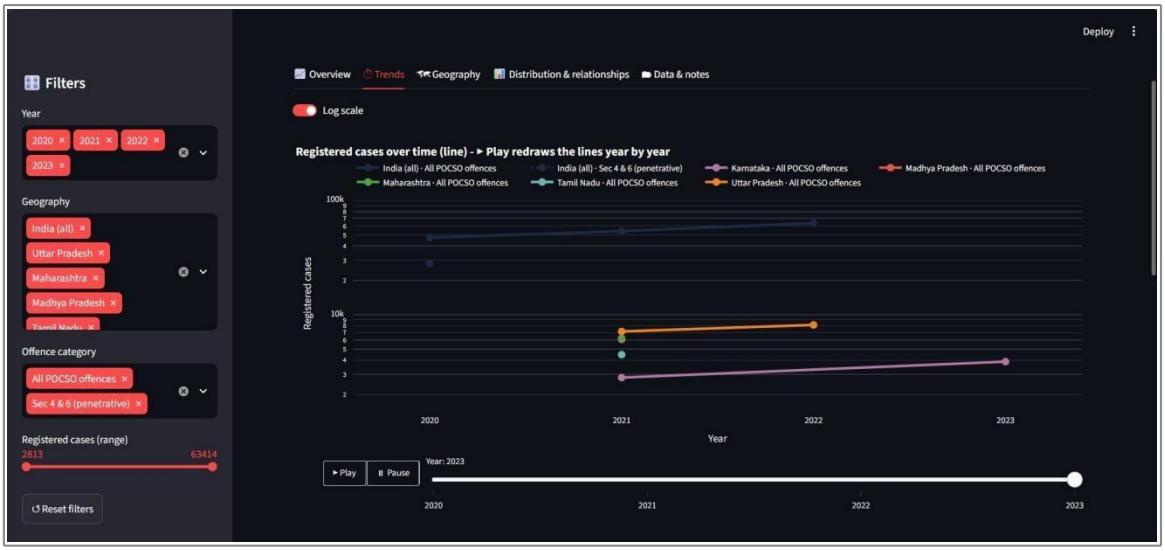


Figure C.5 : Trends tab: animated line chart

Registered cases by geography and offence category on a log scale (the Log scale toggle is on), so the all-India series and the single-state series share one axis. Play redraws the lines year by year; the slider is at 2023.

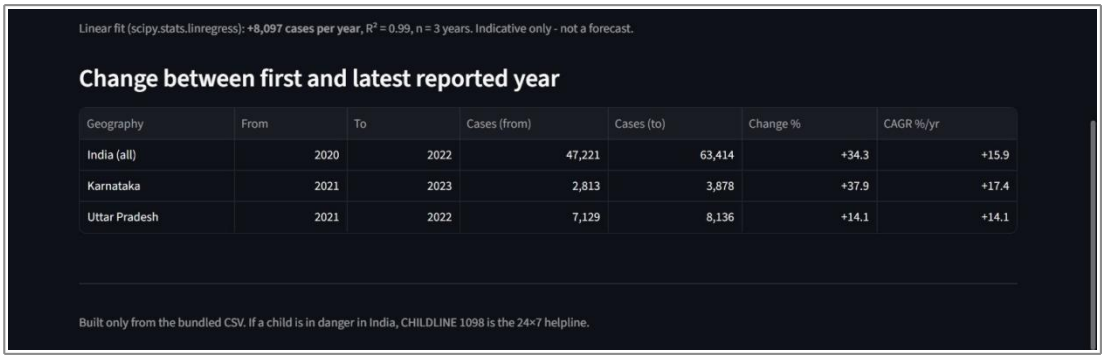


Figure C.6 : Trends tab: linear fit and change table

The scipy linear-fit statement for the all-India series (+8,097 cases per year, RZ = 0.99, n = 3 years, indicative only) and the table of change between the first and latest reported year: India (all) +34.3% (CAGR 15.9%), Karnataka +37.9% (17.4%) and Uttar Pradesh +14.1% (14.1%).

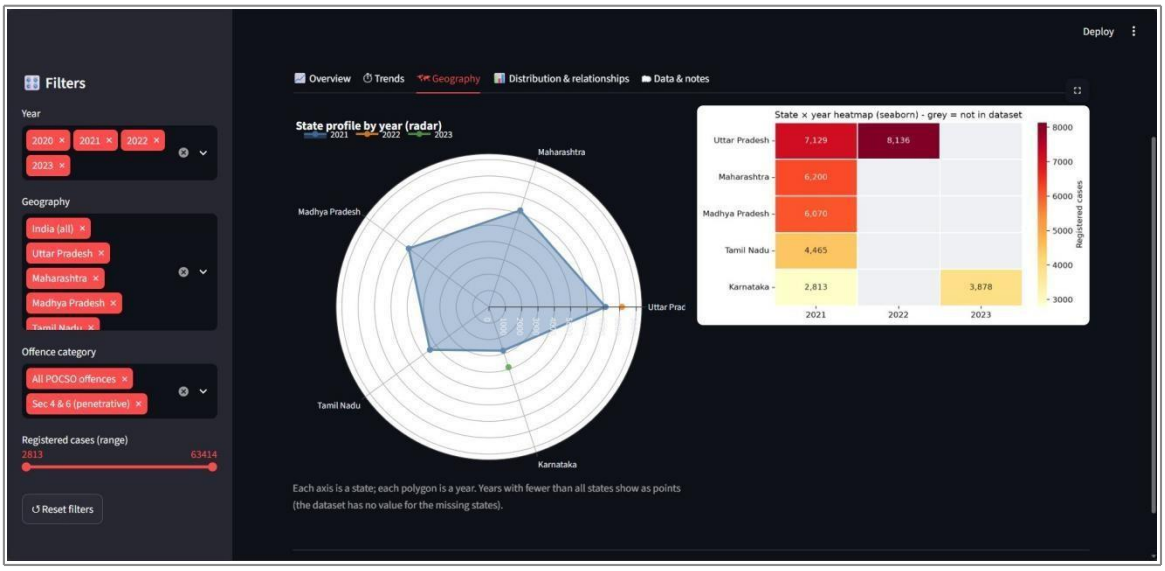


Figure C.7 : Geography tab: radar chart and seaborn heatmap

Left: the radar chart with one axis per state and one polygon per year (2021 is the filled polygon; 2022 and 2023 appear as single points). Right: the state × year heatmap, with grey cells where the dataset has no value.

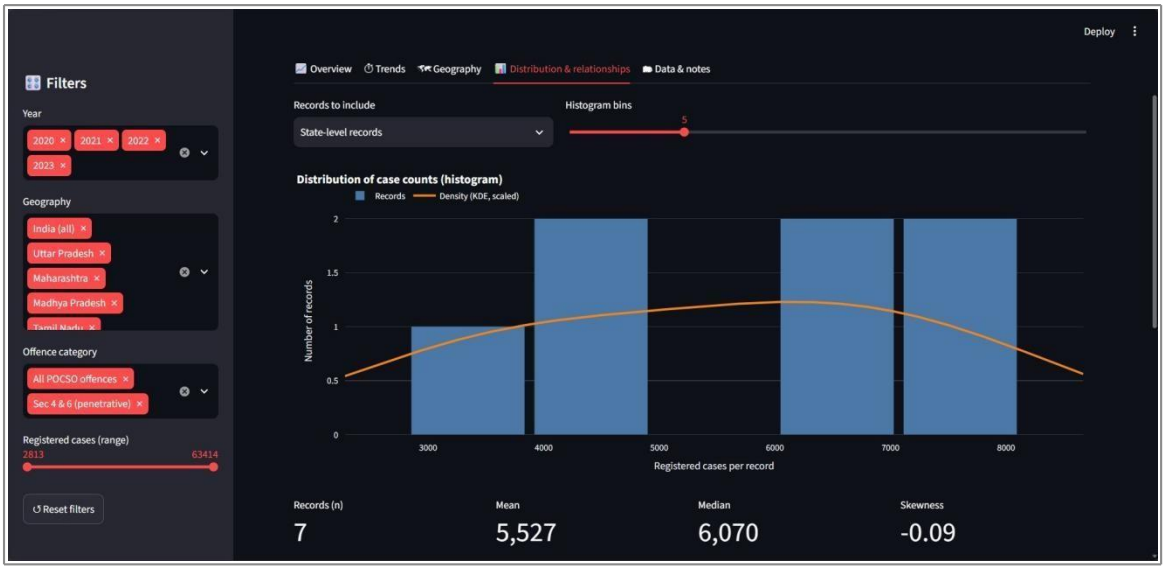


Figure C.8 : Distribution & relationships tab: histogram

The histogram of state-level case counts with the scaled density curve, the record-scope selector, the bins slider (5) and the summary statistics below the chart: n = 7, mean 5,527, median 6,070 and skewness −0.09.

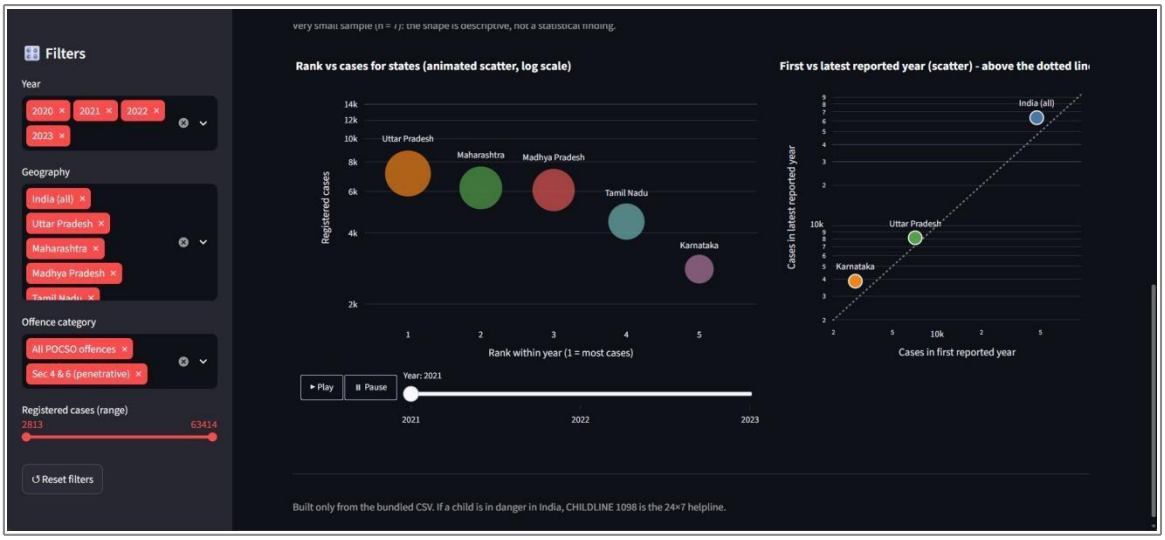


Figure C.9 : Distribution & relationships tab: scatter plots

Left: the animated rank-versus-cases scatter plot on a log scale (frame 2021, bubble size shows cases). Right: first versus latest reported year for India (all), Uttar Pradesh and Karnataka on log–log axes; points above the dotted line show an increase.

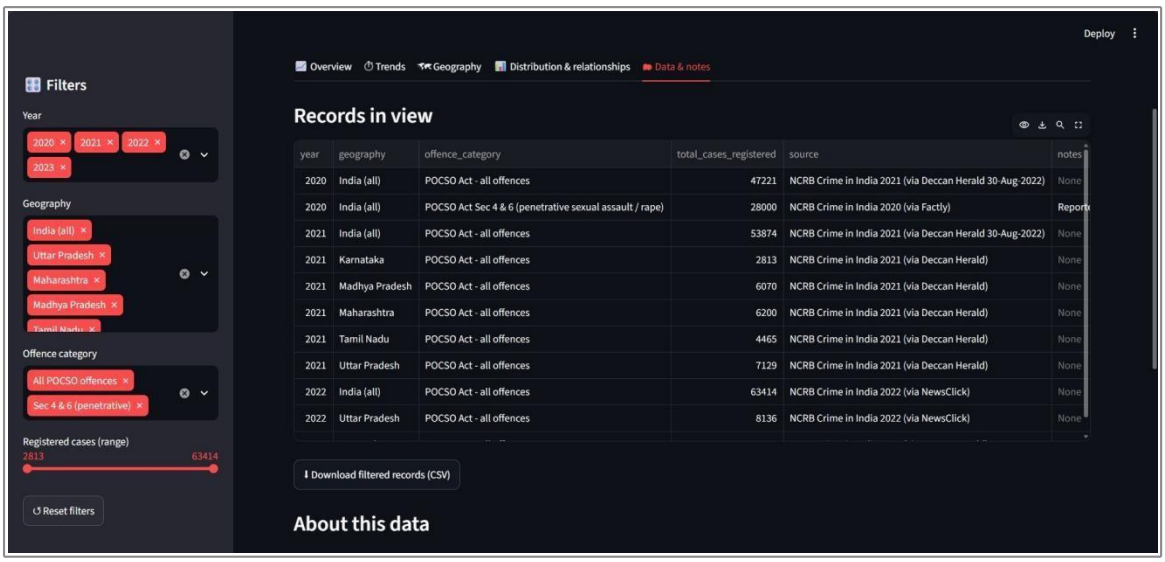


Figure C.10 : Data & notes tab: records in view

The table of records that match the current filters, with year, geography, offence category, total cases registered, source and notes, and the button that downloads the filtered records as a CSV file.

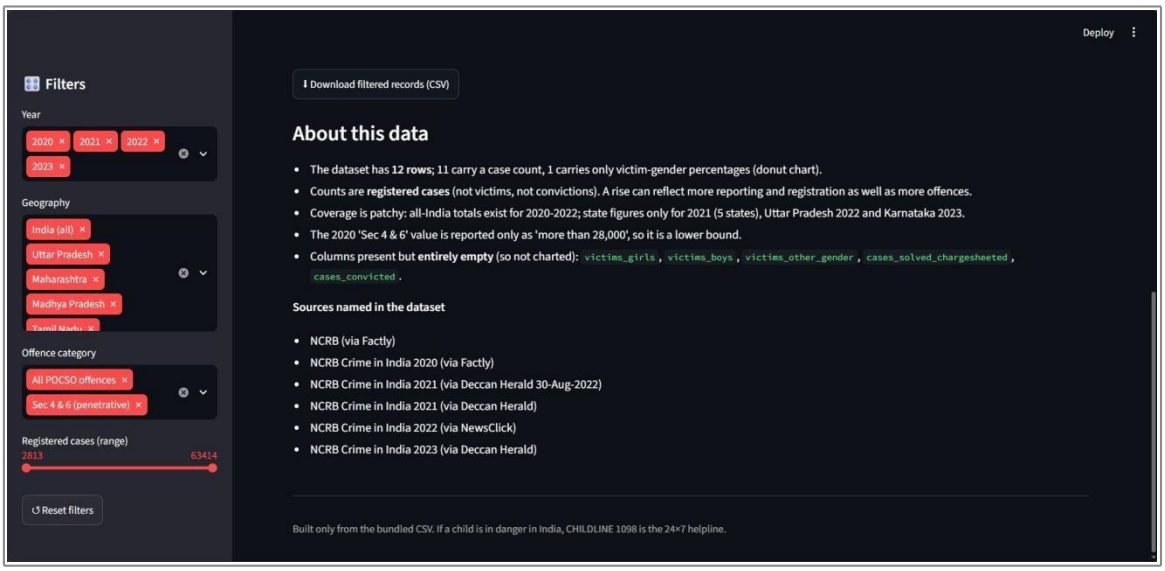


Figure C.11 : Data & notes tab: about this data

The dataset notes: 12 rows, of which 11 carry a case count; registered cases are not victims or convictions; coverage is patchy; the 2020 Sec 4 & 6 value is a lower bound; the empty columns that are not charted; and the sources named in the dataset.